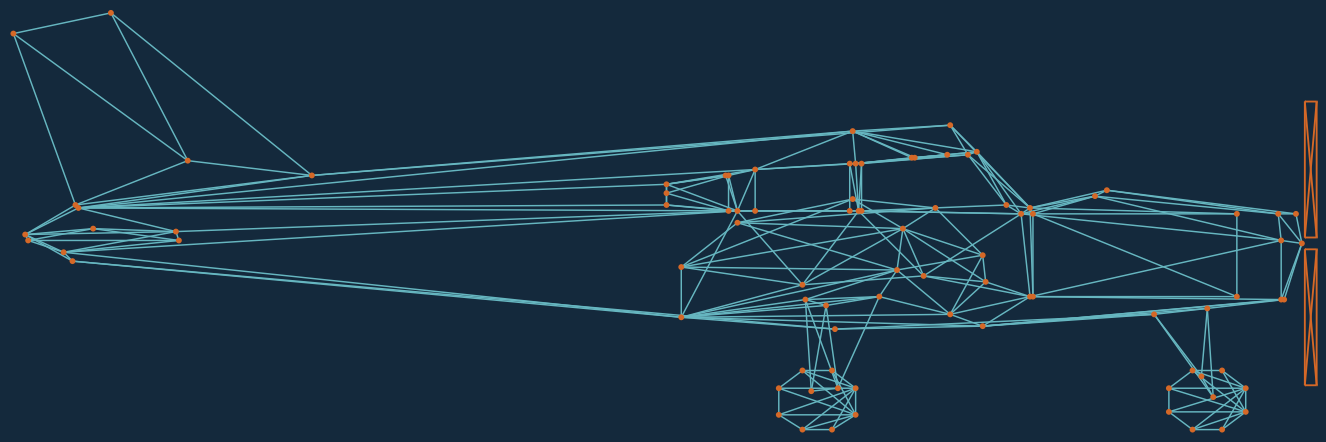
WILDBITS / K2 PC

GPU Programmer's Manual

Renderer | SceneEngine | Point port

Registers, bytecode and working examples



PIPER AIRCRAFT / ORTHOGRAPHIC SIDE VIEW

Revision 1.1

October 27, 2026

Illustrated reference with compact 6809 and BASIC09 examples

Numbered sections - glossary - section and page index

CONTENTS

Contents: reference

Each entry gives its section number and printed PDF page. The PDF outline is also clickable.

Section	Subject	Page
P	Preface	5
1.1	Architecture and programming paths	6
1.2	Coordinates, modes and the viewport	7
1.3	Make the GPU layer visible	8
2.1	Renderer register map	9
2.2	Controls, status and extended registers	10
2.3	Queues, synchronization and diagnostics	11
3.1	Renderer unit encoding and opcode directory	12
3.2	LINE, SPAN and RECT, including 480 rows	13
3.3	CLEAR, HSPLIT, SWAP and FENCE	14
3.4	POLY records and feature flags	15
3.5	Choose corner 0: the fan anchor	16
3.6	Worked example: the leaking L shape	17
3.7	Corner-0 authoring checklist	18
4.1	SETMAT, SETLIGHT and SETPROJ	19
4.2	Mesh memory and MESH command	20
4.3	Depth keys, clipping and transparency	21
5.1	Shading tables and graphics palettes	22
5.2	Flat and Gouraud shading	23
5.3	Image textures: upload and descriptors	24
5.4	UVs, sampling, haze and limits	25

CONTENTS

Contents: scenery and capabilities

Each entry gives its section number and printed PDF page. The PDF outline is also clickable.

Section	Subject	Page
6.1	SceneEngine: register reference	26
6.2	Scene memory, camera and startup	27
6.3	Projection, 480 rows and line width	28
6.4	Scenery bytecode conventions	29
6.5	Scenery opcodes: geometry	30
6.6	Scenery opcodes: texture and flow	31
6.7	Scenery opcodes: variables and branches	32
6.8	Scenery opcodes: origins and selection	33
6.9	Scenery opcodes: legacy services	34
6.10	CPU output records and handoff	35
7.1	Shared point port: registers and commands	36
7.2	Point results, flags and numerical limits	37
8.1	OS-9 access and layer setup	38
8.2	Batch calls, synchronization and IRQ	39
8.3	Feature limits: quick reference	40
8.4	Troubleshooting and source authority	41
8.5	Flight Simulator II: capability showcase	42
8.6	Distance lab: kilometers, miles and map units	43
8.7	Cloud laboratory: shape, shade and reuse	44
8.8	The propeller: a tiny scene hierarchy	45
8.9	Airport flythrough: texture, light and layers	46
8.10	The real aircraft asset, in three views	47
9.1	Glossary: A-M	48
9.2	Glossary: N-Z	49

CONTENTS

Contents: examples and index

Each entry gives its section number and printed PDF page. The PDF outline is also clickable.

Section	Subject	Page
10.1	Small demos: requirements and support	50
10.2	Two small pictures and how to run them	51
10.3	6809: rectangle, then a scene triangle (1/1)	52
10.4	BASIC09: the same two small pictures (1/1)	53
10.5	Add Gouraud shading and an image texture	54
10.5.2	Upload helpers: assembly and BASIC09	55
10.5.3	Exact shade and texture drawing records	56
10.6	Turn the sample into an application	57
I1	Alphabetical index	58
I2	Alphabetical index (continued)	59
I3	Alphabetical index (continued)	60
I4	Alphabetical index (continued)	61
I5	Alphabetical index (continued)	62

PREFACE

Preface

GPU Programmer's Manual documents the WildBits K2 Houston RC28 renderer, SceneEngine scenery processor, and shared point port. Revision 1.1 is dated October 27, 2026. It is a programming reference, followed by compact 6809 and BASIC09 applications with complete shared support source in the companion archive.

The command tables describe the inspected RTL, not a proposed API. Names beginning **SE_** are descriptive mnemonic names used by this manual; the hardware receives their numeric bytes. Renderer names such as **OP_POLY** follow the RTL constants.

> **Read this first**
>
> There are three different streams: renderer units put the opcode in byte 7; scenery lists put their opcode first; point-port commands are written to $FEDE. They are not interchangeable.

Addresses written **$FC:$1800** mean physical 8 KB block $FC, offset $1800. They are not literal CPU addresses. With that block mapped at $6000, the corresponding CPU address is $7800. The examples use grfdrv256 so the driver performs the mapping.

Words are high byte first unless a layout explicitly says otherwise. Hexadecimal values use $. Sizes and field limits are given alongside each feature. Diagrams are explanatory vector illustrations; the sample preview pages use generated command data.

Edition and source boundary

RTL repository snapshot: **b9d1d27d9faa51e9c46987e06e46a755f1772838**, inspected October 27, 2026. This document covers the K2 GPU interfaces. Jr2 has no renderer or SceneEngine. The manual does not claim that an older binary with the same RC number contains every later repair.

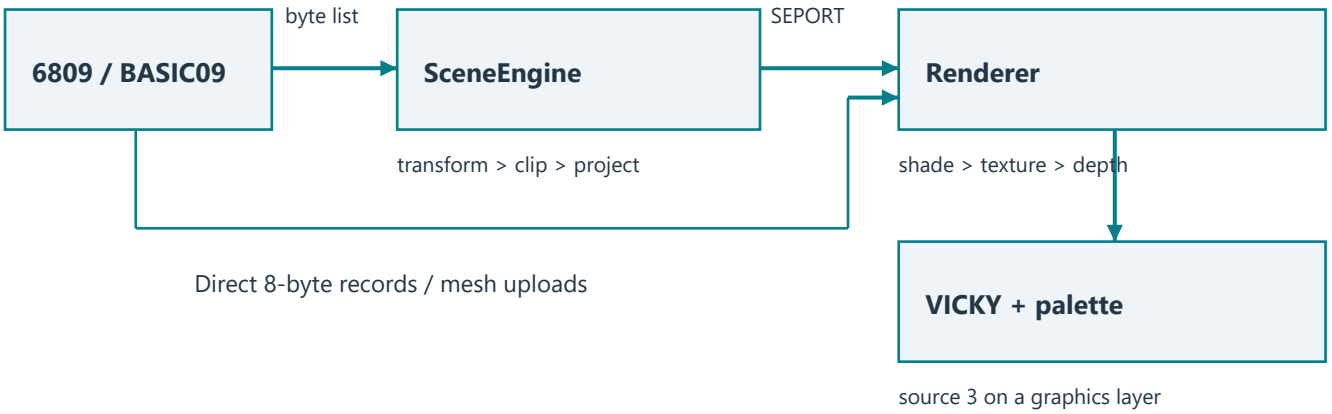
How to use this book

Start with Sections 1-2 for visibility and register access. Use Sections 3-5 for CPU records, meshes, shading and textures. Section 6 is the scenery instruction reference. Sections 7-8 cover the point port, OS-9 calls and limits. The glossary is Section 9. The corner-0 tutorial is in 3.5-3.7 and the Flight Simulator II showcase in 8.5-8.10. Short application listings and additional rendering recipes are in Section 10; the index gives section and page numbers.

Examples are educational programs for an idle, live terminal. Hardware execution is explicitly distinguished from source-level and host-model checks in Section 10.1. This manual does not modify or build a core.

1.1

Architecture and programming paths



Path	Input	What hardware does
Direct renderer	8-byte units	2D primitives, polygon fans, uploaded meshes, texture/shade lookups and depth test.
SceneEngine	Byte-coded scenery list plus private variable RAM	Walks flow control; transforms, clips and projects geometry; sends drawing records to CPU or renderer.
Point port	Ground, 3D or camera-space points	Transforms or projects individual points and returns words to the CPU. It shares arithmetic with SceneEngine.

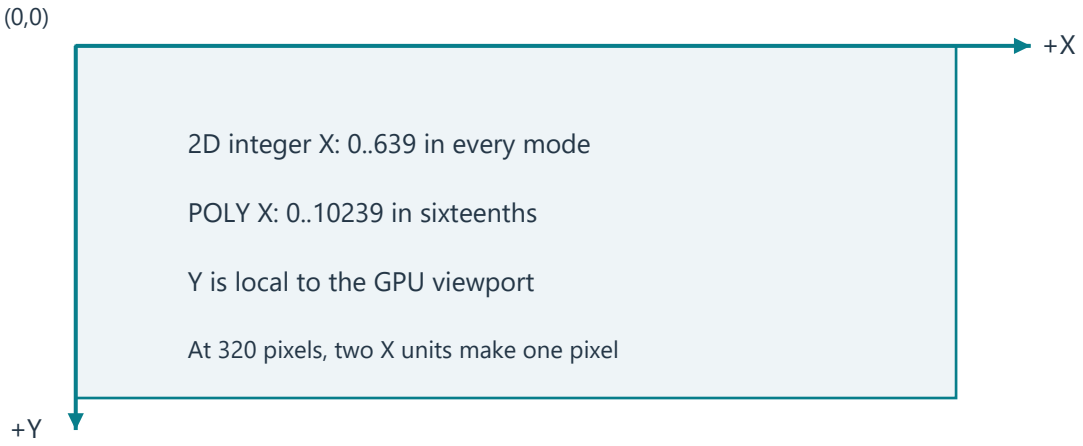
The renderer has its own front/back color buffers and depth memory. SceneEngine RAM, GPU asset memories, and system SRAM are separate address spaces. Uploading a mesh does not upload scenery, and neither is an OS-9 memory allocation.

Limits

One global renderer and one global SceneEngine serve the K2. They are not per-process contexts. Coordinate transforms and shading are fixed-function hardware; there are no programmable vertex or fragment shaders. Rendering cost depends on primitive count, area, overdraw, clipping and textures; no universal frame-rate guarantee follows from the clock rate.

1.2

Coordinates, modes and the viewport



Mode	Control	Color / active rows
640 x 240	GPU_CTRL.MODE = 0	4 bits per pixel; MODE480 has no effect.
320 x 240	MODE = 1, MODE480 = 0	8 bits per pixel.
320 x 480	MODE = 1, MODE480 = 1	8 bits per pixel; both viewport fields have a ninth bit.

VIEW_ROW0 positions the already-rendered viewport in display-row space. Drawing Y starts at zero within that viewport. **VIEW_ROWS** limits rendering and clearing; zero or a value above the mode maximum selects the full height. The front-buffer mode and height are latched at SWAP; layer/palette selection and viewport origin are live controls.

2D LINE/SPAN/RECT use integer X in 640-dot space even in 320 mode. POLY uses signed 16-bit X,Y in sixteenths, also in 640-dot X space. Mesh projection produces the same POLY coordinates. Negative POLY positions may be clipped by the rasterizer; integer LINE coordinates must already fit.

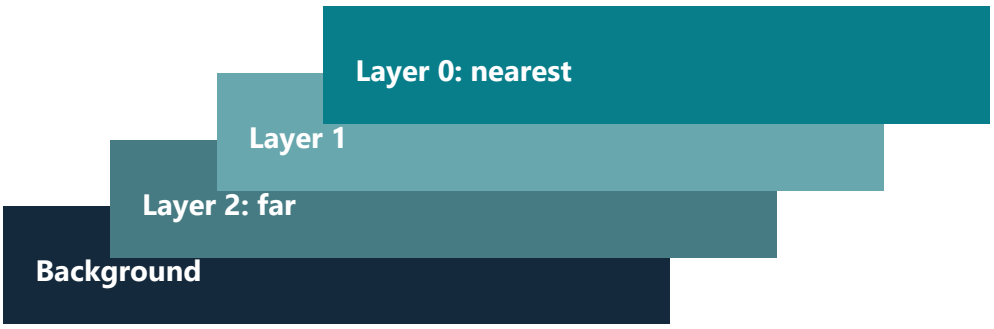
Limits

There is no 640 x 480 GPU mode. A 320 x 480 frame uses 153,600 color bytes; each 240-row mode uses 76,800. Two color frames are provided. Depth is 16 bits per pixel: up to 153,600 pixels. The implemented mode uses GPU memory, not sprite pattern storage; sprites are not disabled by MODE480.

1.3

Make the GPU layer visible

GPU can occupy any layer whose source is 3.



Pixel 0

reveals what is beneath.

Requirement	Register / operation	Meaning
GPU enabled	$FC:$1800 bit 0	EN must be one.
Layer source = 3	$FFC2 bits 2:0 or 6:4; $FFC3 bits 2:0	Layer 0, 1 or 2 selects the GPU. Layer 0 is nearest.
Graphics path enabled	$FFC0 bits 2 and 3	Graphics and bitmap gates must both be enabled.
Front frame exists	Submit SWAP and wait	Drawing changes the back frame; it does not immediately appear.
Useful palette	$FC:$1808 and graphics CLUT	Framebuffer values are palette indices, not RGB.
Text policy	$FFC0 bits 0-1; $FFC1 bit 4	Text can cover the graphics. The examples temporarily turn text off.

For OS-9 use **SS.Layer ($8E): X=0, Y=3** to select GPU on layer 0, and **SS.MCR ($8C): X=$0C, Y=0** for graphics+bitmap without text, in the 60 Hz mode used by the examples. Get SS.MCR first and restore both returned bytes afterward. These calls maintain the terminal mirror as well as the live registers.

Limits and ownership

Source 3 is visible only while GPU EN is set. Source 7 is a blank layer. Pixel value zero reveals a lower layer or background. Keep the demo terminal live: GPU state is global and the OS driver does not save a complete GPU context on terminal switches. Use 60 Hz / 480-line video timing for the full viewport examples.

2.1

Renderer register map

All offsets below are inside physical block **$FC**. R = readable; W = writable. A Q mark means the write queues an ordered renderer operation and consumes FIFO capacity. Reset values refer to hardware reset, not GPU_CTRL.RESET.

Offset	Name / access	Fields, reset and behavior
$1800	GPU_CTRL RW / Q	EN, MODE, ZEN, DITHER, RESET, AUTOCLEAR, TEXEN, SEPORT; reset $00. See 2.2.
$1801	GPU_STATUS R / W1C	Read status. Write ones clears error bits 2:0 only.
$1802-03	FIFO_FREE R	0..512 units. Read high first; it latches the low byte.
$1804 / 05	SWAP_COUNT / FENCE_COUNT R	8-bit completion counters, modulo 256; hardware reset 0.
$1806 / 07	VIEW_ROW0 / VIEW_ROWS RW	Low 8 bits of origin / height. Origin live; height Q. Reset 0 / 240.
$1808	GPU_LUT RW	Bits 2:0 CLUT select; bits 6:4 16-color group. Mask $77; reset 0. See 5.1 for implemented limits.
$1809	CLEAR_COLOR RW / Q	AUTOCLEAR color; reset 0. Explicit CLEAR uses its record byte 6.
$180A-C	UP_ADDR RW	24-bit upload address, high/middle/low; reset 0.
$180D	UP_DATA W / Q	Uploads one byte at UP_ADDR, then increments address. Reads return 0.
$180E-F	SE_YMUL RW	Unsigned scale: routed Y16 = PY16 * value >> 10. Reset $0400 = 1.0.
$1810-11	PROBE_ROW RW / Q	Bit 15 enable; bits 8:0 row. Reset 0.
$1812-13	PROBE_DOT RW / Q	Pixel X in actual mode: 0..319 or 0..639. Reset 0.
$1814	PROBE_COLOR R	Last successful renderer write covering the enabled probe; not a memory read port.

2.2

Controls, status and extended registers

Bit	GPU_CTRL ($1800)	GPU_STATUS ($1801)
7	SEPORT: route scenery drawing to GPU	BUSY: queued, active or adapter work
6	TEXEN: allow textured fragments	SWAP_PENDING
5	AUTOCLEAR after SWAP	FRONT: current color buffer selector
4	RESET: write 1; reads 0	FIFO_EMPTY (does not mean rendering is finished)
3	DITHER: shade-level ordered dither	IDLE: inverse of BUSY
2	ZEN: enable polygon depth flags	MESHERR (sticky)
1	MODE: 1=320 / 8-bit, 0=640 / 4-bit	BADREC (sticky)
0	EN: show GPU source on VICKY	OVERFLOW: a CPU unit was dropped (sticky)

Offset	Name	Meaning / limits
$1815	HAZE_CTRL	Bit 0 enabled; reset 0; Q.
$1816-17	HAZE_START	16-bit depth key threshold; Q, high then low.
$1818-19	HAZE_SCALE	16-bit haze multiplier; Q, high then low.
$181A	HAZE_COLOR	Final palette index for haze replacement; Q.
$181B	MODE480	Bit 0 enables 480 rows only with MODE=1; Q; reset 0.
$181C	VIEW_ROW0_HI	Bit 0 is origin bit 8; live; reset 0.
$181D	VIEW_ROWS_HI	Bit 0 is height bit 8; Q; reset 0.
$181F	GPU_ID	Read $34; it identifies the interface, not a unique RC build.
$1840-7F	RECORD window	8 aliases of the same 8-byte latch; byte offset modulo 8.
$181E, $1820-3F	Reserved	Read 0; writes ignored. $1880-$18FF is sprite mirror storage.

GPU RESET aborts current work, drains queued units, clears sticky errors and reloads drawing controls from bus shadows. It is not a clear of asset RAM or both framebuffers. Do not send a new stream until IDLE is observed; configure and explicitly CLEAR before drawing.

2.3

Queues, synchronization and diagnostics

Work is executed in FIFO order.



SWAP publishes at vertical blank; FENCE reports earlier work complete.

The CPU must keep room for uploads, register writes and complete records.

Drain SceneEngine OUT even when drawing records use SEPORT.

The FIFO holds **512 units of 8 bytes**, not 512 complete polygons. Every UP_DATA write consumes a unit. Ordered control writes consume units too. A textured 8-vertex POLY consumes 17 units; SETMAT consumes 5. Check FIFO_FREE before writes. It may report zero while the SceneEngine adapter owns pending work, even when physical FIFO slots remain.

Do not interleave a CPU command inside a SceneEngine-produced command or between another producer's header and continuation units. Finish and drain SceneEngine first, then send CPU records. A single producer may stream a large list in capacity-checked batches. Never wait for IDLE halfway through a multi-unit command: the renderer is waiting for its remaining units.

Operation	Use	Limit
SWAP	Publish the back frame at vertical blank. Count increments then.	Counter wraps at 256. It may wait for video timing.
FENCE	Observe completion of prior records without publishing.	Count wraps at 256; not a separate memory upload mechanism.
PROBE	Set row/dot, then draw or CLEAR and wait. Read the observed color.	Sees subsequent writes only; no arbitrary front-frame readback.
Errors	Read status bits 2:0; clear with W1C.	A dropped or malformed command needs recovery, not just clearing the flag.

Polling and IRQ

BUSY includes active parsing, drawing and adapter work. FIFO_EMPTY alone is insufficient. GPU SWAP/FENCE completion and SceneEngine stop pulses share INT_PENDING_1 bit 1 ($FE21 mask $02). See 8.2 for interrupt ownership and acknowledgments. Use bounded waits.

3.1

Renderer unit encoding and opcode directory

byte 0 payload	byte 1 payload	byte 2 payload	byte 3 payload	byte 4 payload	byte 5 payload	byte 6 payload	byte 7 OPCODE
$1840	$1841	$1842	$1843	$1844	$1845	$1846	$1847



Write 0 through 6 first. The byte-7 write commits one FIFO unit.

A continuation also has a tag in byte 7: $FE.

Tag	Mnemonic	Units	Payload summary
$00	OP_LINE	1	X0, X1, Y0, Y1, color
$01	OP_SPAN	1	X0, X1, Y, color
$02	OP_RECT	1	X0, X1, Y0, Y1, color
$03	OP_CLEAR	1	Z-clear flag and color
$04	OP_HSPLIT	2	Two signed points and two colors
$08	OP_SWAP	1	No operands
$09	OP_FENCE	1	No operands
$10	OP_POLY	1+N or 1+2N	Flags, material, texture, count, vertices, optional UVs
$20	OP_SETMAT	5	35 payload bytes: matrix, translation, scale
$21	OP_SETLIGHT	2	14 payload bytes: light, ambient, diffuse
$22	OP_SETPROJ	2	14 payload bytes: focal, center, near, clip shift
$28	OP_MESH	1	Face-entry offset/count, vertex-entry base, material override
$FE	T_CONT	1	Seven bytes continuing a multi-unit record
$FF	T_NOP	1	Ignored; does not count as a continuation

Tags $F0 (upload) and $F1 (register write) are internal ordered units generated by the bus registers. Use UP_DATA and the documented registers. Unknown tags set BADREC. Unused payload bytes should be zero.

3.2

LINE, SPAN and RECT, including 480 rows

Byte	LINE $00	SPAN $01 / RECT $02
0-1	X0, unsigned high byte first	X0
2-3	X1, unsigned high byte first	X1
4	Y0 low byte	Y / Y0 low byte
5	Y1 low byte	Ignored for SPAN / Y1 low byte
6	Color index	Color index
7	$00	$01 / $02

480-line encoding: when MODE=1 and MODE480=1, bit 15 of the X0 word carries Y0 bit 8; bit 15 of X1 carries Y1 bit 8. The renderer removes those bits before checking X. Byte 7 remains the opcode. The hardware does not double LINE rows.

```
packed_x0 = x0 | ((y0 & 0x100) << 7)
packed_x1 = x1 | ((y1 & 0x100) << 7)
// Example LINE from (40,300) to (600,479), color 15:
80 28  82 58  2C DF  0F 00
```

240 rows: each row repeats

row 0
row 0
row 1
row 1
row 2
row 2
row 3
row 3
row 4
row 4

480 rows: each row is distinct

row 0
row 1
row 2
row 3
row 4
row 5
row 6
row 7
row 8
row 9

Primitive	Edge behavior	Limits
LINE	Inclusive endpoints; Bresenham-style stepping.	X 0..639 and Y < effective height; bad endpoint sets BADREC; no line clipping.
SPAN / RECT	Sorts endpoints; inclusive coverage.	Right/bottom clipped to limits; wholly out-of-range start draws nothing. No depth test/write.
All three	X becomes floor(X/2) in 320 mode.	4-bit mode stores color low nibble. Zero is transparent during display.

grfdrv256 GpuRec.480 is a software conversion flag for old 240-row LINE records. It doubles Y and emits additional coverage. Leave that flag clear when records already use native 480-row coordinates.

3.3

CLEAR, HSPLIT, SWAP and FENCE

CLEAR $03 - one unit

```
00 00 00 00 zz 00 cc 03
zz bit 0: also clear Z to zero; cc: color
```

Clears the back frame's effective VIEW_ROWS and optionally its depth. It also updates an enabled probe inside the cleared view. It does not clear textures, meshes, shade tables, or the displayed front frame.

HSPLIT $04 - two units

Unit	Bytes 0..6	Byte 7
Header	X0hi X0lo Y0hi Y0lo X1hi X1lo 00	$04
Continuation	Y1hi Y1lo colorA colorB 00 00 00	$FE

Coordinates are signed integer 640-space X and viewport rows. The directed line divides every active row into two colors. At sample center (x,y), A is used where **(Y1-Y0)(x-X0) - (X1-X0)(y-Y0) > 0**, B otherwise. A horizontal left-to-right line puts A above it. HSPLIT fills the complete viewport and ignores depth.

SWAP $08 and FENCE $09 - one unit each

```
00 00 00 00 00 00 00 08   ; publish at vertical blank
00 00 00 00 00 00 00 09   ; ordered completion point
```

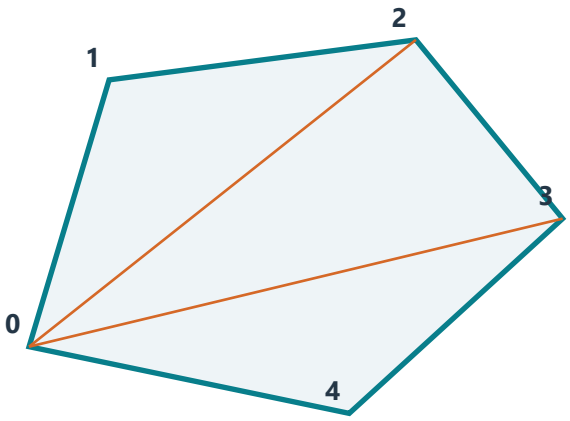
With AUTOCLEAR, SWAP starts a color-and-Z clear of the new back frame using CLEAR_COLOR. The count changes at the exchange; IDLE may arrive later after the automatic clear. Without AUTOCLEAR, clear explicitly before each new frame when old depth/color should not persist.

Limits

HSPLIT is a two-color fill, not a gradient or a clipped polygon. All 2D primitives bypass Z. Submit sky/ground before depth-tested objects, then overlays last. SWAP/FENCE payload bytes have no effect; clear them for repeatable streams.

3.4

POLY records and feature flags



Convex fan

(0,1,2) (0,2,3) (0,3,4)

3-8 vertices per POLY

X,Y: signed 1/16 units

Shade: 0-255

Unit(s)	Bytes 0..6	Tag
Header	flags material texture N 00 00 00	$10
N vertices	Xhi Xlo Yhi Ylo Qhi Qlo shade	$FE
N UVs if TEXTURED	Uhi Ulo Vhi Vlo 00 00 00	$FE

Send **all vertex units first, then all UV units**. N is 3..8. POLY triangulates a fan anchored at vertex 0. Supply a convex polygon or triangulate in software; holes and arbitrary concave outlines are not defined by this interface. Vertex shade is 0..255; without GOURAUD the first vertex supplies the flat shade.

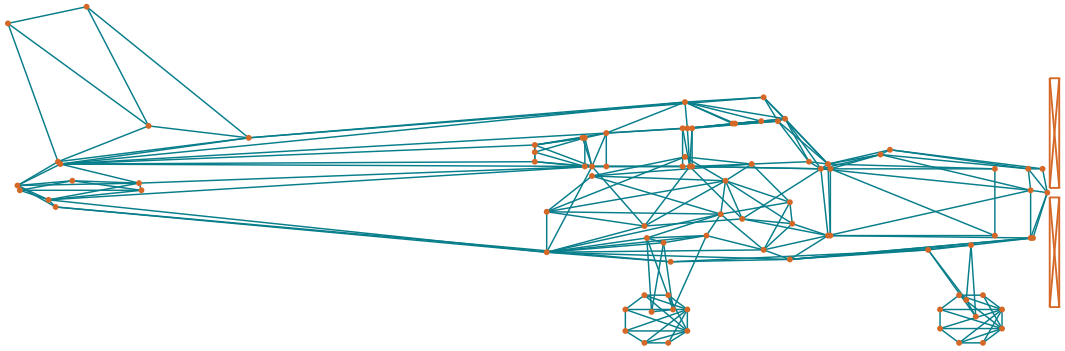
Mask	Flag	Effect
$01 / $02	ZTEST / ZWRITE	Depth compare / write, each also gated by GPU_CTRL.ZEN.
$04	GOURAUD	Interpolate vertex shade. Clear means flat first-vertex shade.
$08	TEXTURED	Parse UV units; texture sampling also requires TEXEN.
$10	SCREENDOOR	Discard alternating pixel/row parity: 50% coverage.
$20	DIRECT	Use material/texel as final palette index; bypass shade LUT.
$40	CULL	Reject non-positive signed screen-space area.
$80	Reserved	No implemented feature. Write zero.

With Y increasing downward, positive standard signed area is clockwise. Avoid DIRECT when demonstrating Gouraud: DIRECT bypasses the shading lookup even if GOURAUD is set. TEXTURED controls packet length even when TEXEN is off.

3.5

Choose corner 0: the fan anchor

A POLY is not a general outline-fill operation. The renderer makes triangles **(0,1,2), (0,2,3), ...**, always using the first vertex as the anchor. SceneEngine FILL follows the same fan through its adapter, even when a long outline is divided into several POLY packets.



PIPER AIRCRAFT / ORTHOGRAPHIC SIDE VIEW

The Piper is already a triangle mesh. Its explicit three-corner faces do not depend on one anchor for the whole wing or aircraft.

Outline	Best choice
Convex polygon	Any boundary corner is valid. Keep the perimeter order; never sort points independently by X or Y.
Concave polygon	A usable anchor must see the entire interior: every generated fan triangle must remain inside and tile it without overlap. Some outlines have no usable boundary corner.
Wing with a notch or inset	Split into convex panels or author explicit triangles. Do not force the complete silhouette into a single fan.
Hole or self-crossing contour	Corner rotation does not solve it. Supply a properly triangulated surface.

A broad corner is a quality heuristic, not a rule

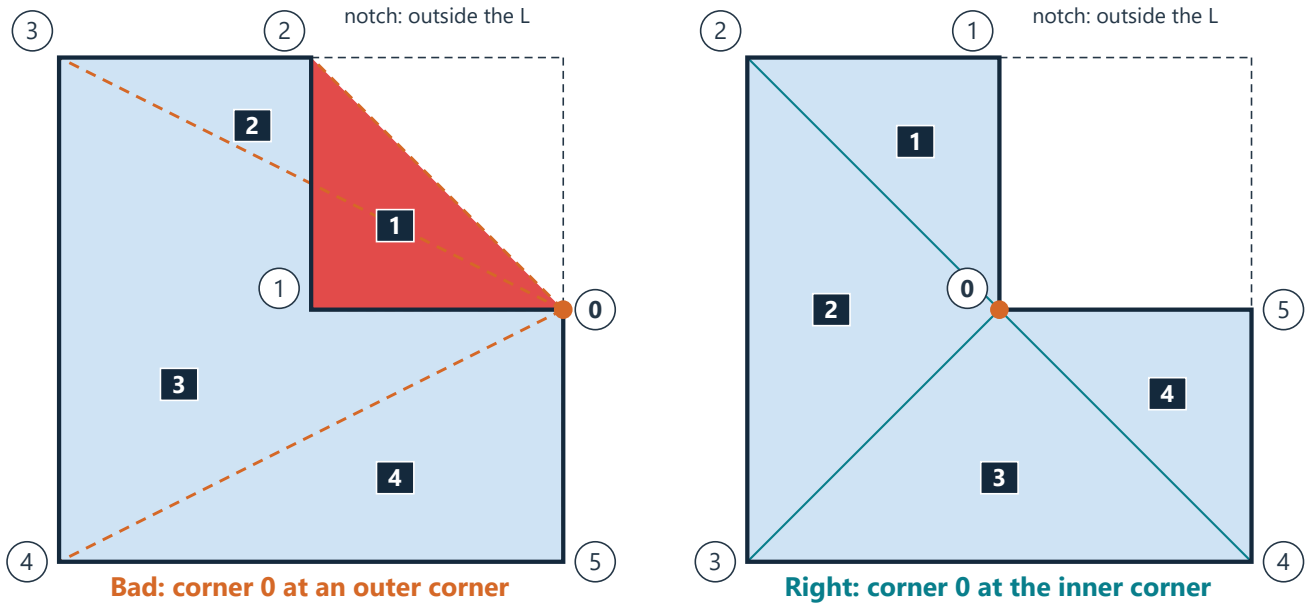
For an already valid fan, prefer a stable anchor that avoids needlessly thin triangles. A wing root can be a better choice than a sharp tip, but geometry decides. Corner 0 does not have to be the topmost, leftmost or nearest vertex. Changing coordinates to hide an invalid fan is not the remedy.

What this prevents

A bad fan can cover space outside a concave outline: a geometric fill leak. This tutorial does not explain wrong palette colors, colored dots on odd scanlines, or a broken CLUT write path.

3.6

Worked example: the leaking L shape



Bad: corner 0 at an outer corner
triangle 1 lies outside the L (red)

Right: corner 0 at the inner corner
triangles 1-4 cover the L exactly

How corner 0 fans through the polygon. The renderer starts at corner 0 and walks the outline in the order the corners were sent. Triangle 1 is corners 0-1-2, triangle 2 is 0-2-3, triangle 3 is 0-3-4, and the last of an N-corner polygon is 0-(N-2)-(N-1). Each triangle reuses the previous one's far edge, so the triangles sweep around corner 0 like the blades of a fan. The renderer never chooses its own lines: it cannot skip a corner or pick a shorter diagonal. The triangles add up to the polygon only when every straight line from corner 0 to another corner stays inside the outline.

Right example. Corner 0 is the L's inner corner. Triangles 1-4 are 0-1-2, 0-2-3, 0-3-4 and 0-4-5. Every line from corner 0 stays inside the L, so the four triangles cover the L exactly, with no overlap and nothing outside.

Bad example. The same six corners, the list rotated by one, so corner 0 is an outer corner beside the notch. The next two corners in the list are 1 and 2, the notch's own corners, so triangle 1 (0-1-2) lies entirely in the notch, outside the L (red). Triangle 2 (0-2-3) also crosses the notch on its way to corner 3; only from triangle 3 on do the triangles stay inside. The four triangles cover the L plus a wedge of the notch, so what is drawn is not the intended polygon: every pixel in that wedge is a fill leak. The orange dashed lines are the fan's lines from corner 0.

The corners. In the bad order, in POLY units (X 0-639, Y 0-239): 0 (440, 120), 1 (260, 120), 2 (260, 30), 3 (80, 30), 4 (80, 210), 5 (440, 210). Use X and Y multiplied by 16 in a POLY vertex. Sent as listed, corner 0 is the outer corner and the polygon leaks. Sent from corner 1 (1-2-3-4-5-0), the inner corner is the new corner 0 and the fan is the right example. No coordinate changed.

Guaranteed simple repair

Start the list at the inner corner, as in the right example, or send the four triangles 0-1-2, 0-2-3, 0-3-4 and 0-4-5 of the right example as separate POLYs with N=3, or as four mesh faces. An offline triangulator is preferable for general concave assets.

The safe concave example illustrates the fan condition; arbitrary concave polygons remain unsupported. A convex decomposition is the general authoring recommendation.

3.7

Corner-0 authoring checklist

Step	Do this
1. Inspect in the face plane	Walk the perimeter exactly once. Remove accidental duplicate points and reject self-intersections. Use a planar face for predictable projection.
2. Test the fan	Draw all diagonals from the candidate first corner. All triangles must be inside, with no overlap or inverted triangles. If none works, split the face.
3. Preserve winding	Rotate A-B-C-D to C-D-A-B; do not reorder it to C-A-D-B. With CULL set, the renderer accepts positive signed screen area (clockwise with Y downward).
4. Rotate complete vertices	Move X, Y, Q, shade and UV together. For POLY, rotate the UV continuation sequence in the same way. For scenery, place UV before its corresponding MOVE3 or DRAW3.
5. Keep shading stable	Flat POLY shade comes from vertex 0. Preserve that intended value. Changing diagonals can change Gouraud and UV interpolation; do not change the anchor every frame.
6. Test clipping and motion	Check near-plane and screen-edge crossings. Clipping may create new corners and change the first output vertex. Convex faces remain the robust input.
7. Check shared edges	Adjacent pieces must share identical positions and matching attributes where continuity is intended. Avoid ad hoc offsets that replace leaks with cracks.

Minimal authoring pattern

```
Safe order:   0 1 2 3 4 5   (corner 0 at the inner corner)
Explicit fan: 0 1 2 | 0 2 3 | 0 3 4 | 0 4 5

Rotate records, not just coordinates:
  vertex = (X16, Y16, Q, shade, U8.8, V8.8)
```

Limits still apply

A direct POLY carries 3-8 vertices. SceneEngine adapter capacity is 32, including generated clip vertices. Backface culling removes triangles; it does not repair concavity. Dithering smooths color transitions, not the silhouette of a thin wing.

4.1

SETMAT, SETLIGHT and SETPROJ

For these commands concatenate the seven payload bytes from the header and each $FE continuation into p[0..]. Fields can cross a unit boundary; do not insert padding between fields.

Command	Payload offsets	Encoding
SETMAT $20 5 units	0-17: M0..M8	Nine signed Q1.15 words, row-major for the renderer mesh transform.
	18-21, 22-25, 26-29: Tx,Ty,Tz	Three signed 32-bit values in 24.8 camera units.
	30: scale; 31-34: zero	Low nibble scale 0..15, left shift of model coordinates before translation.
SETLIGHT $21 2 units	0-5: Lx,Ly,Lz; 6: ambient; 7: diffuse	Signed Q1.15 direction; ambient/diffuse unsigned 0..255. Bytes 8-13 zero.
SETPROJ $22 2 units	0-3: FX,FY; 4-7: CX,CY	FX,FY unsigned 16-bit; CX,CY signed 16-bit. All in screen sixteenths.
	8-11: NEAR; 12: clip shift	NEAR signed 32-bit 24.8, clamped to at least 1. Shift low nibble 0..15; byte 13 zero.

Transform and projection

```
camera = (((M * model) << scale) >> 7) + T
X16 = CX + floor(FX * cameraX / cameraZ)
Y16 = CY + floor(-FY * cameraY / cameraZ)
clip_distance = max(1, NEAR >> clip_shift)
```

The transform produces 24.8 camera coordinates; model positions are signed 16-bit integers. Q1.15 $7FFF approximates +1; $8000 is -1. Mesh lighting transforms normals by M, computes max(0, dot(normal, light)), scales it by diffuse, adds ambient, and clamps to 0..255.

Limits

The hardware does not normalize normals or light vectors. Use a rotation/uniform-scale setup and normalized Q1.15 normals; general nonuniform transforms need software normal correction. SETMAT and SETPROJ affect MESH, not already-projected POLY or SceneEngine geometry. Initialize all three commands before your first MESH; do not rely on power-up defaults.

4.2

Mesh memory and MESH command

MESH $28 - one unit

```
faceOffsetHi faceOffsetLo faceCountHi faceCountLo
vertexBaseHi vertexBaseLo materialOverride 28
```

Offsets and indices are **16-byte entry numbers**, not byte addresses. Upload byte offset = entry number * 16 into target $400000. Faces and vertices share the same 64 KB memory; place their ranges so they do not overlap.

Entry	Byte offsets	Fields
Vertex (16 bytes)	0-5	Signed model X,Y,Z words.
	6-11	Signed Q1.15 Nx,Ny,Nz words.
	12-15	Unsigned 8.8 U,V words.
Face (16 bytes)	0-5	Unsigned i0,i1,i2 words; actual entry = vertexBase + index.
	6 / 7 / 8 / 9	Material / texture ID / POLY flags / zero padding.
	10-15	Signed Q1.15 face normal Nx,Ny,Nz.

Faces are triangles. The engine transforms their vertices, shades them, clips against the near plane and a guard rectangle, projects, then rasterizes the resulting fan. GOURAUD uses vertex normals; flat shading uses the face normal. Material override zero uses each face's material; a nonzero override replaces it.

Limits

4,096 total mesh entries. Every faceOffset + face number and vertexBase + index must be below 4096. A bad vertex index sets MESHERR and skips the face; an out-of-range face entry ends that draw. A count of zero draws nothing. The guard band is X,Y in [-16384,26624] sixteenths; final viewport clipping is separate. There is no skeleton, instancing table, animation controller, mipmap system or automatic normal generation.

A practical update loop

Upload vertices/faces once. Each frame clear the back frame, set the camera/object matrix and light, send MESH for each object, then SWAP. Change T for motion; change M for rotation. Sharing vertex entries reduces upload memory; the hardware may still transform a shared vertex separately for each face.

4.3

Depth keys, clipping and transparency

Property	Exact behavior
Key meaning	16-bit custom floating inverse-depth key, not IEEE half float. Larger unsigned value is nearer.
Compare	A fragment passes when new Q >= stored Q. Equal-depth fragments may overwrite in stream order.
Clear	Z=0 is farthest. ZTEST and ZWRITE need both the primitive flag and GPU_CTRL.ZEN.
Packing	Q28 = min(2^28-1, floor(NEAR * 2^28 / cameraZ)); encode below.
Interpolation	Packed vertex keys are decoded to fixed inverse depth before triangle interpolation, then packed again.
Transparency	Framebuffer index 0 reveals lower layers. Texture texel 0 discards color and depth writes. SCREENDOOR discards half the samples.

```
encode(q):
  if q >= (1<<28)-1: return $FFFF
  if q < (1<<14):    return q >> 2
  p = floor(log2(q))
  return ((p-13)<<12) | ((q>>(p-12)) & $0FFF)
```

SETPROJ's clip shift moves the mesh near clipping plane closer without changing the key reference. Geometry nearer than NEAR saturates at the nearest key, reducing depth discrimination there. POLY already contains projected vertices and Q: it performs no 3D near-plane clipping for the caller.

Limits

A direct color-zero polygon may still write depth even though the displayed index is transparent. Use texel-zero discard or omit ZWRITE when that is not intended. Depth is finite precision; coplanar decals require deliberate order/flags. There is no alpha blending, stencil buffer, depth readback API or programmable compare function.

5.1

Shading tables and graphics palettes

32 shade-table entries for one material



level 0 level 31

Example: map those 32 levels into a 16-color CLUT ramp



The ramp is software data. The core does not invent its colors.

There are **two distinct lookups**. The GPU shade table maps (mode, material, shade level) to a color index. VICKY's graphics CLUT maps that index to RGB. Uploading one does not initialize the other.

```
shade_offset = (mode320 << 13) | (material << 5) | level
upload_address = $800000 + shade_offset
level = min(31, (shade + dither_bias) >> 3)
```

The shade table contains 16,384 bytes: two mode banks, each 256 materials x 32 levels. Without DITHER, bias=0. With it, bias=Bayer4[y mod 4][pixelX mod 4] >> 1, from 0..7. Software chooses whether 32 levels become 32 distinct colors, a 16-step ramp, or any other mapping.

Palette layer	Layout / range	Practical rule
GPU_LUT	Bits 2:0 select; 6:4 group	The compositor has four distinct CLUT banks: use select 0..3. Encodings 4..7 alias after the address is truncated.
Graphics CLUT	$FD:$1000 + 1024*bank + 4*index	256 entries per bank, bytes B,G,R,A. RGB channels are 8 bits; A is not an alpha-blending control.
640 / 4-bit group	Index = (group << 4) | nibble	Group 0..7 exposes 16-color slices 0..127; color nibble 0 is transparent.
320 / 8-bit	Full byte index; group ignored	Index 0 is transparent; 1..255 are visible colors.

The four graphics CLUTs are shared display resources. Under OS-9 save/restore the entries you own using SS.Gpu access to block $FD. SS.Palet selects a CLUT; it does not upload RGB entries. Avoid slot 7 when mapping $FD: its offsets $1D00-$1FFF overlap fixed CPU I/O, including part of CLUT 3.

5.2

Flat and Gouraud shading

Mode	POLY setup	MESH setup
Flat	GOURAUD clear. First vertex shade supplies the polygon's shade.	GOURAUD clear. Face normal supplies light response.
Gouraud	Set $04. Supply each vertex's 0..255 shade.	Set $04 in face flags. Supply normalized vertex normals.
DIRECT	Set $20: material/texel is final index; no shade-table lookup.	Same flag behavior; computed lighting does not affect final index.
Textured + shaded	Set $08 and TEXEN, clear DIRECT. Texel selects a material row.	Same, with UVs from vertex entries.

Light calculation

```
normal_view = (M * normal_model) >> 15
dot = max(0, (normal_view . light) >> 15)
shade = clamp(ambient + ((diffuse * dot) >> 15), 0, 255)
```

Gouraud interpolates the scalar shade across each projected triangle. It does not interpolate RGB. A missing or incorrectly loaded shade row can therefore produce black or unrelated colors even when the geometry and lighting are correct.

16-step ramp recipe

Reserve CLUT entries 16..31 and load a dark-to-light RGB ramp. For material 1 in 320 mode, upload 32 shade-table bytes at $802020: entry k = 16 + floor(k/2), k=0..31. A triangle with shades 0,128,255 then spans the ramp. GPU_CTRL.DITHER chooses neighboring levels spatially; it does not increase the number of physical palette entries.

Limits

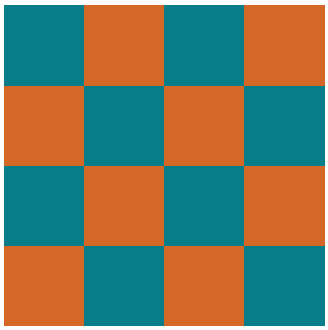
There is one ambient and one directional diffuse light. No specular term, per-pixel normals, bump mapping or physically based lighting is implemented. Shade is 8-bit but the lookup selects 32 quantized levels. Dither is ordered 4 x 4, not error diffusion or antialiasing. Normals are supplied by the application; smoothing across faces requires suitable shared/averaged normals.

The supplied examples use DIRECT for their first shapes so no shade upload is required. Section 10.5 adds a shaded triangle and texture setup explicitly.

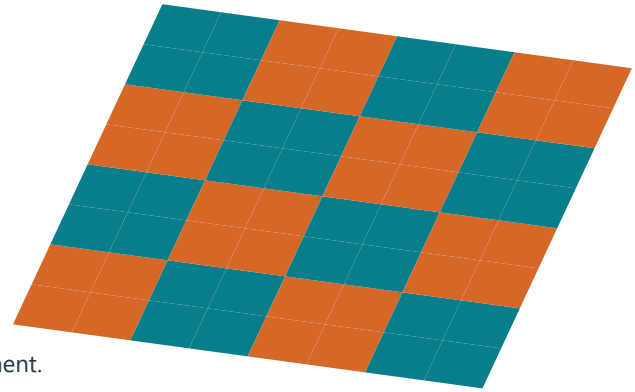
5.3

Image textures: upload and descriptors

Indexed image: one byte per texel



UVs place the image on the polygon



Nearest-neighbor sampling; texture index 0 discards the fragment.

Target	Address	Capacity / format
Texel image	$000000 + byte offset	262,144 bytes. One unsigned material/color index per texel, row-major.
Descriptor	$C00000 + 4*textureID	256 descriptors, 4 bytes each.
Descriptor bytes 0-2	base address, high first	Only low 18 bits used; address wraps at 256 KB.
Descriptor byte 3	Vclamp Uclamp hhh www	width=2^(3+www), height=2^(3+hhh). Bit 6 clamps U; bit 7 clamps V.

```
// Texture 0, 8 x 8 image starting at texel byte 0:
UP_ADDR = $C00000
UP_DATA = $00, $00, $00, $C0   // clamp both axes
// Then upload 64 texels at $000000.
```

With DIRECT, nonzero texels are final palette indices. Without DIRECT, texels are material IDs and are passed through the shade table. A texel of zero is always discarded when sampling is enabled, so it cannot be an opaque material 0.

Image conversion

Decode PNG/BMP/etc. on the host, quantize to the intended index palette, resize/pad to power-of-two dimensions, and upload raw indices. The core does not decode image file formats or compressed images. Upload the matching graphics palette and, for shaded textures, every used material's shade row.

5.4

UVs, sampling, haze and limits

U and V are unsigned 8.8 texel coordinates. $0100 means 1.0 texel, not a normalized full-image coordinate. Sampling uses bits 15:8 as the integer coordinate; fractional bits support interpolation. POLY sends one UV unit per vertex after all position units. Mesh vertices store U,V in bytes 12..15.

Feature	Behavior	Limit
Filtering	Nearest neighbor; no bilinear blend.	No mipmaps, anisotropic filtering or texture compression.
Perspective	Interpolate U*Q and V*Q; divide at 16-pixel segment ends, interpolate between.	Perspective correction is segmented, not an exact divide at every pixel.
Wrap	Integer coordinate AND (dimension-1).	Power-of-two dimensions only.
Clamp	Integer coordinate is limited to dimension-1.	Unsigned coordinates; no negative-coordinate clamp behavior.
Dimensions	Descriptor can encode 8..1024.	U/V integer part is only 8 bits. Use 8..256 for fully addressable images; larger descriptors do not add coordinate range.
Memory bounds	Base + row*width + column, low 18 bits.	No allocation guard: overlapping images or wrap can sample unrelated texels.

Distance haze

```
h = min(16, ((HAZE_START - Q) * HAZE_SCALE) >> 16)
// Only if 0 < Q < HAZE_START; otherwise h = 0.
// Replace with HAZE_COLOR if h > Bayer4[y & 3][x & 3].
```

Enable HAZE_CTRL bit 0. Haze replaces selected covered fragments with one color index; it is not RGB blending. It acts on rasterized polygons, including DIRECT polygons, after texture-zero rejection. Q=0 never hazes. Integer 2D LINE/SPAN/RECT and HSPLIT bypass this polygon stage.

Limits

Haze distance is measured in the packed inverse-depth key, not linear world distance. The threshold pattern is 4 x 4 even if shade DITHER is off. Texture transparency, screen-door coverage, palette index zero and haze are different mechanisms; combine them deliberately.

6.1

SceneEngine: register reference

SceneEngine walks byte-coded scenery in its own RAM. Its CPU registers occupy **$FC:$1780-$178F**. Words are high byte first. Drawing can be returned to the CPU or routed directly to the renderer with GPU_CTRL.SEPORT.

Offset	Read	Write
$1780 / 81	OUT high / latched low. Reading high pops a word.	Reserved.
$1782	STATUS: bit 7 OUT ready, 6 busy, 5:3 state, 2:0 = 3.	1 GO, 2 GO-AT, 3 RESUME, 4 ABORT.
$1783	OUT word count, saturated at 255.	Bit 0 abort + flush OUT; bit 1 clear error flags.
$1784 / 85	Upload address high / low.	Set upload/readback address.
$1786	RAM byte at address, then increment.	RAM byte at address, then increment.
$1787 / 88	Last stop/list address P, high / low.	Resume address LP, high / low. Does not select the GO start.
$1789	Stopped opcode or fault code.	Reserved.
$178A	Errors: bit 0 busy access; bit 1 OUT underflow.	Reserved.
$178B	View byte index.	Index 0..11 for valid view bytes.
$178C	View byte, then index advances modulo 16.	View byte, then advance; indices 12..15 ignored.
$178D / 8E	Line half-width LW, high / low.	Unsigned 8.8 list units; zero selects thin lines.
$178F	Zero.	Reserved.

State bits 5:3	Meaning
0 IDLE	Reset/aborted.
1 RUN	Executing or completing output.
2 CPU	Stopped for a CPU-handled instruction; RESUME allowed.
3 DONE	Top-level END/RETURN completed.
4 FAULT	Invalid memory access (1) or return-stack overflow (2).

RAM/view writes require the engine not busy. OUT holds 1,024 words and can stall execution when full. Count saturation is not a capacity limit. Drain OUT during execution, including when SEPORT is on.

6.2

Scene memory, camera and startup

Range / field	Meaning
$0000-$1FFF	8 KB variable/context RAM, with dirty-byte writeback. Geometry and legacy operations also use this RAM as workspace.
$4000-$7FFF	16 KB scene-list RAM. CPU uploads; not an extension of the system CPU map.
$2000-$3FFF, $8000+	Unmapped. A model may have bank extensions which this RTL does not implement.
$007B-$007C	Big-endian starting list pointer for both GO and GO-AT.
$0068-$0079	Nine signed Q1.15 matrix words. Camera component j = dx*M[j] + dy*M[3+j] + dz*M[6+j], with fixed-point scaling.
$000C-$0011	Local eye X,Y,Z: signed 16-bit words for 3D point transforms.
$0000-$000B	Global eye X,Y,Z: three 32-bit values used by origin/range instructions.
$0042-$004A	Ground transform base: three signed 24-bit values, derived by origin processing.
$0178-$0179	Polygon fill color word; PEND takes its low nibble. Line color is separate, at $00EE/$00EF.
$08B3 + 8*slot	Saved point slots 0..30; indices above 30 alias slot 29. Do not use these as general-purpose variables.

Start and stop

GO copies radio activity bytes $028C to $028D, $02A3 to $02A4, and $020E to $020F. **GO-AT** skips those copies. Both read the start at $007B, latch the matrix, reset the 16-level return stack, clear $0282/$007A, and start untextured. The line-width register survives a GO.

RESUME works only in CPU state: set LP through $1787/$1788, then issue command 3. It preserves the saved call stack. ABORT alone does not flush queued OUT words; use $1783 bit 0 for abort plus flush.

Context ownership

The variables are a compatibility ABI, not 8 KB of freely allocatable application storage. The walker writes geometry scratch, clip state, counters and radio state. The minimal example initializes the fields it uses; a full application must preserve or reconstruct the context for its chosen instructions. There is no hardware instruction-count timeout. CPU software must abort a runaway list.

6.3

Projection, 480 rows and line width

View byte offsets	Word	Units / effect
0-1 / 2-3	CX / CY	Screen center in sixteenths.
4-5 / 6-7	SX / SY	Unsigned legacy projection scale words. Approximately 4*scale*cameraAxis/cameraZ.
8-9 / 10-11	XMAX / YMAX	Maximum projected X/Y in sixteenths; projection clamps to these bounds.

```
Approximate projection (fixed rounding is implementation-specific):
  PX16 = CX + 4*SX*cameraX/cameraZ
  PY16 = CY - 4*SY*cameraY/cameraZ
```

View	CX, CY, SX, SY, XMAX, YMAX
Reset (legacy 147 rows)	5120, 1176, 1280, 294, 10239, 2351
Full 240 rows	5120, 1920, 1280, 480, 10239, 3839
Full 480 rows	5120, 3840, 1280, 960, 10239, 7679

Select view byte 0 at $178B, then write the twelve bytes to $178C. These parameters belong to SceneEngine and the point port. Renderer SETPROJ is a different transform interface for MESH.

240 rows: each row repeats



480 rows: each row is distinct



For 320 x 480, also set GPU_CTRL.MODE, MODE480 bit 0 and height=480 ($01E0). Either project into a 480-row view with SE_YMUL=$0400, or retain the 240-row view and use SE_YMUL=$0800. Do not double both. Integer renderer LINE/SPAN/RECT records are never automatically doubled by this register; use their explicit ninth Y bits.

Line width limits

LW=0 produces the adapter's thin-line geometry. Nonzero LW is a half-width in 1/256 list units: full ribbon width is about 2*LW/256. The scenery engine builds a ground-plane ribbon using the endpoint XZ direction. This is not a screen-pixel pen width or a general join/cap renderer; unsupported endpoint combinations fall back to the thin path.

6.4

Scenery bytecode conventions

Each instruction begins with its opcode, followed immediately by parameters. **The high opcode bit is masked off.** All sizes below include the opcode. The mnemonic names in this manual are descriptive names; use the displayed numeric byte unless your assembler defines those symbols.

Notation	Definition
w / s16	Two bytes, high first. Signedness follows the field; positions and relative displacements are signed 16-bit.
b	One byte.
rel	Signed 16-bit displacement from P, the address of this instruction's opcode. Target = P + rel, wrapping to 16 bits.
addr	A 16-bit address in SceneEngine variable/list space, not a CPU logical address.
X,Y,Z	List/model coordinate integers; camera matrix/origin convert them.
slot	Saved point slot 0..30. Larger values map to slot 29.
CPUSTOP	Writeback and a CPU-stop record are emitted. Software handles the operation, chooses the resume address and issues RESUME.

A small polygon

```
25 00 09 01 78       SE_SETW $0009,$0178 ; fill 9
2F                   SE_PBEGIN
01 FF A0 FF C0 01 00 SE_MOVE3 -96,-64,256
02 00 60 FF C0 01 00 SE_DRAW3  96,-64,256
02 00 60 00 40 01 00 SE_DRAW3  96, 64,256
02 FF A0 00 40 01 00 SE_DRAW3 -96, 64,256
29                   SE_PEND
79                   SE_END
```

PBEGIN starts polygon collection. MOVE3 supplies the first point and DRAW3 supplies following points. PEND clips/projects and emits the fill. Convex polygons are the safe input contract: the adapter triangulates a fan, not a general concave shape or a polygon with holes.

Geometry limits

The adapter stores up to 32 vertices after the scenery path produces them. Extra vertices are ignored when that buffer is full; there is no separate overflow status for this. Leave headroom for clipping, and split large shapes yourself. It emits renderer fans in groups of 3..8 vertices. SceneEngine FILL uses a 4-bit color with DIRECT, so it does not automatically use the renderer's Gouraud shade table.

An unrecognized opcode enters CPU state. In particular, $3A/$3B banked-list extensions in a host model are not implemented scenery instructions in the inspected RTL.

6.5

Scenery opcodes: geometry

Byte / mnemonic	Parameters	Bytes / behavior
$00 SE_DOT3	Xw,Yw,Zw	7. Transform and emit one projected point.
$01 SE_MOVE3	Xw,Yw,Zw	7. Set current 3D point; first vertex when collecting a polygon.
$02 SE_DRAW3	Xw,Yw,Zw	7. Draw from previous point or append polygon vertex.
$06 SE_SLINE	endpointD w, endpointX w	5. Legacy packed screen endpoints, byte X/row fields; view mapping emits X endpoint first, D endpoint second.
$12 SE_COLOR	color b	2. Set low-nibble line color. Does not select polygon fill.
$1B SE_COLORFF	None	1. Set line color to $F.
$1C SE_COLORDUSK	None	1. Set line color $E if RAM $013F bit 0 is clear; otherwise preserve color.
$29 SE_PEND	None	1. End/fill polygon using $0179 low nibble, or finish the saved line path when outside a polygon.
$2B SE_LONGLINE	X0w,Z0w,X1w,Z1w	9. Ground line divided into 16 intervals; draw eight alternating intervals. Current LW applies.
$2F SE_PBEGIN	None	1. Begin polygon collection.
$31 SE_SLOT3	slot b,Xw,Yw,Zw	8. Transform/store 3D point in a slot.
$32 SE_MOVESLOT	slot b	2. Move to stored point.
$33 SE_DRAWSLOT	slot b	2. Draw to stored point / append vertex.
$35 SE_DOTSLOT	slot b	2. Emit stored point.
$40 SE_MOVEG	Xw,Zw	5. Move to transformed ground point.
$41 SE_DRAWG	Xw,Zw	5. Draw to ground point / append vertex.
$42 SE_SLOTG	slot b,Xw,Zw	6. Transform/store ground point.

Thin 3D lines and dots use depth test without depth write in the routed adapter. 3D polygons use test and write. A polygon containing a ground-marked vertex takes the decal path without depth test/write. SLINE bypasses depth. All depth behavior also depends on global ZEN.

6.6

Scenery opcodes: texture and flow

Byte / mnemonic	Parameters	Bytes / behavior
$36 SE_TEX	textureID b	2. Enable textured polygon state. Select an uploaded renderer texture.
$37 SE_NOTEX	None	1. Disable textured polygon state.
$38 SE_UV	U w,V w	5. Set sticky unsigned 8.8 texel coordinates for subsequent vertices.
$09 SE_CALLPTR	pointerAddress w	3. Read a list pointer from that address. Zero skips call; otherwise push return P+3 and jump.
$0B SE_GOTO	rel w	3. Unconditional branch to P+rel.
$11 SE_NOP	None	1. No operation.
$18 SE_CALL	rel w	3. Push P+3 and branch to P+rel; update legacy call depth at $0270.
$19 SE_RETURN	None	1. Return from current call; top-level return ends execution.
$79 SE_END	None	1. Clear radio active flags $02A4/$028D/$020F, then return. At top level this ends the list.

Texture sequence

```
SE_TEX textureID
SE_PBEGIN
SE_UV 0,0        ; precedes the corresponding point
SE_MOVE3 ...
SE_UV $0700,0
SE_DRAW3 ...
SE_UV $0700,$0700
SE_DRAW3 ...
SE_UV 0,$0700
SE_DRAW3 ...
SE_PEND
SE_NOTEX
```

The routed adapter emits DIRECT textured polygons: nonzero texels are final color indices. GPU_CTRL.TEXEN must also be set. The engine does not load an image or descriptor for SE_TEX, and UVs do not advance automatically. GO/GO-AT clear texture state.

Flow limits

The return stack is 16 levels, shared by ordinary calls and runway sublist calls. END inside a sublist returns to its caller; it is not a global abort. There is no unconditional hardware halt timeout and no supported bank-switch instruction. Keep all targets within the mapped list/context RAM and bound loops in CPU software.

6.7

Scenery opcodes: variables and branches

Byte / mnemonic	Parameters in order	Bytes / decision
$1A SE_COPYW	sourceAddr w,destAddr w	5. Copy word; destination must be variable RAM.
$25 SE_SETW	value w,destAddr w	5. Write literal word. Value comes BEFORE destination.
$20 SE_RANGE1	rel w; addr w,low w,high w	9. Fall through if signed value is in inclusive range; otherwise branch.
$21 SE_RANGE2	rel w; two (addr,low,high) triples	15. All signed ranges must pass; otherwise branch.
$22 SE_RANGE3	rel w; three (addr,low,high) triples	21. All signed ranges must pass; otherwise branch.
$23 SE_BITS	rel w,addr w,maskHigh b,maskLow b	7. Fall through if any masked word bit is nonzero; otherwise branch.
$28 SE_CMP	condition b,rel w,addrA w,addrB w	8. Branch if condition true: 0 equal, 2 signed greater, 4 signed less. Other conditions CPUSTOP.
$04 SE_CULLSLOTS	rel w,slot bytes...,terminator	Variable, 4+number of slot bytes. Terminator has bit 7 set. Branch if the common $F8 outcode rejects all listed points; otherwise continue after terminator.

These tests read words from SceneEngine memory, not the 6809's current process map. Relative branches are based on the opcode address. Treat address wrap as an error in your list generator; the engine will fault only when the resulting access is outside its map.

Comparison compatibility output

SE_CMP also produces a two-byte compatibility writeback at address $D0A3. That address is a host compatibility result, not a mapped SceneEngine RAM address. An OS-9 consumer must interpret it deliberately; never blindly store all returned writeback addresses into its process memory.

Limits

A range check is signed 16-bit and inclusive. It does not support floating point, unsigned ordering or 32-bit values. CULLSLOTS must contain a terminating byte; it has no separate length field. The clip outcodes are a coarse rejection test, not a replacement for clipping intersecting geometry.

6.8

Scenery opcodes: origins and selection

Byte / mnemonic	Parameters	Bytes / behavior
$07 SE_ORIGIN32	mode b,X32,Y32,Z32	14. Set local origin from three 32-bit positions. Each position is LOW 16-bit word first, HIGH word second; each word itself high byte first.
$24 SE_ORIGIN16	mode b,Xw,Yw,Zw	8. Each coordinate is the high word of a 32-bit world position. Derive eye/origin/ground values.
$13 SE_NEAR	rel w,Zw,Xw,Rw	9. Fall through when absolute signed-16 differences from global eye Z and X high words are both strictly less than R; otherwise branch.
$14 SE_NEARALT	rel w,Zw,Xw,Rw	9. NEAR test plus eye-Y high word < R. Uses $018A when RAM $0139 is nonzero, otherwise global eye at $0004.
$0D SE_AREA	block b,rel w,unused b,slot b	6. Write block to $0249, P+rel to $0247, slot to $024A. Continue if RAM[$024C+slot] equals block; otherwise CPUSTOP after those writes.

Origin scale

```
mode < $04 or mode >= $84 : shift = 4
mode = $04                : shift = 8
mode = $06                : shift = 12
other modes               : shift = 16
```

Origin instructions implement the legacy scenery coordinate convention; they are not a renderer SETMAT equivalent. The same mode byte affects conversion from global eye/origin values to the local point and ground transform state. Supply the full global eye context before using them.

Limits

NEAR/NEARALT use legacy 16-bit subtraction/wrap behavior, not a Euclidean sphere or full 32-bit world distance. R is also the altitude threshold in NEARALT. AREA does not read a disk or load a bank: it asks the host to satisfy that condition and resume. A small standalone scene can avoid these compatibility operations entirely, as the example does.

6.9

Scenery opcodes: legacy services

Byte / mnemonic	Parameters	Bytes / effect
$03 SE_RUNWAY	heading b,length w,width w	6. Uses heading plus RAM $014A/$0148, computes runway variables, and calls template list $0DE9. That template must be uploaded into variable RAM.
$05 SE_NAV05	id w,6 data bytes	9. If $020F active, clear $020E. If ID equals $09A5, write bytes [d0,0,d1,d2,d3,0,d4,d5] to $0293-$029A and set $09A4=1.
$1D SE_NAV1D	id w,8 data bytes	11. If $028D active, clear $028C. ID matching $0291 writes $029B-$02A2 and flag 1; matching $028F writes $0293-$029A and flag 2 at $028E.
$1E SE_ATC	rel w,id w	5. If $02A4 active, clear $02A3; ID $FFFF sets $0203=$FFFF, and a matching ID enters CPUSTOP. Otherwise clear $0282/$007A and branch to P+rel.

These opcodes expose compatibility behavior for a flight-simulator scenery stream. Their names describe the observed operations, not a complete application-level radio, runway or air-traffic-control API.

Limits

RUNWAY is not a self-contained primitive: the template at $0DE9 and its supporting context are required. NAV05/NAV1D update state only when their activity flags are set; GO-AT does not initialize those flags for you. ATC can require CPU handling. The manual's house example uses none of these services.

Unsupported instructions

All opcode values not listed in Sections 6.5-6.9 enter CPU state, after their opcode high bit is masked. The core does not silently skip an unknown instruction because it does not know its length. Software must determine the length and semantics before choosing a resume address.

Do not treat a host reference model as a list of implemented RTL features. The inspected model contains banked scenery experiments absent from this hardware revision. This reference lists the hardware decoder, including its CPU-stop behavior.

6.10

CPU output records and handoff

OUT is a stream of **16-bit words**. The first word carries type in its high byte and flags/color/length in its low byte. Read $1780 then $1781 for every word. Do not mistake these records for the renderer's eight-byte FIFO units.

Header	Following words with SEPORT=0	Meaning
$01ff	X16,Y16; then U,V if ff bit 2	Vertex. Bit 0 marks first vertex.
$02cc	textureID if cc bit 4	Fill; low nibble color; bit 4 textured.
$03cc	Xa16,Ya16,Xb16,Yb16	3D line in projected coordinates.
$04cc	X16,Y16	Projected dot. Two coordinates, not an old packed XR field.
$05cc	Xa16,Ya16,Xb16,Yb16	Screen line.
$10ll	address, ceil((ll+1)/2) data words	Writeback: ll+1 bytes (1..16), high byte first; odd final low byte is zero. Runs stay within one 16-byte block.
$20oo	P	CPU stop at opcode oo, list address P.
$22ee	P	Fault ee at P: 1 address, 2 stack.
$3000	instructionCount	End, count modulo 65,536.

When SEPORT=1

Drawing records go to the GPU adapter, not the CPU. Internally VTX gains Q after X,Y and flag bit 1 marks ground; LINE3 gains Qa,Qb; DOT gains Q. CPU writeback, stop, fault and END records still arrive at OUT. The driver must drain them while execution runs, or a full FIFO can stall the engine.

At each stop, dirty-variable writeback runs precede the stop record. Apply those to your application's explicit context representation if needed. The example owns a disposable context and drains/discards its writeback; a simulator that expects updated variables must parse them.

Completion sequence

Start GO/GO-AT, repeatedly drain OUT, inspect status, handle CPU stops or faults, and continue until DONE with no pending output. Only then enqueue frame SWAP after the routed drawing. GPU FIFO empty alone does not prove that SceneEngine or the adapter has finished.

7.1

Shared point port: registers and commands

CPU address	Write	Read
$FED8 / D9	Ground (G) word high / low; low commits.	Common result FIFO high / latched low.
$FEDA / DB	3D point (P) word high / low; low commits.	Same result FIFO, not a separate P queue.
$FEDC / DD	Camera/projection (Q) word high / low; low commits.	Same result FIFO.
$FEDE	Command byte.	Status: ready 7, busy 6, input nearly full 5, any error 4, bit 3=0, bits 2:0=5.
$FEDF	Bit 0 reset FIFOs/errors; bit 1 clear errors.	Result word count, saturated at 255.

Command	Input words	Operation
$01 LOADM	9	Load signed Q1.15 matrix in SceneEngine ordering.
$02 LOADEYE	3	Load signed eye X,Y,Z words.
$03 LOADBASE	5	Nine bytes of signed 24-bit ground-base components; final low byte padding.
$10 ABSOLUTE	0	P input subtracts eye before transform.
$11 DELTA	0	P input is already relative to eye.
$20-$2F MARK	0	Return marker $A500 OR low command nibble after earlier queued work.
$30 QUERY	0	Return $5050,$0100,errorFlags,completedPointCount.

Send command operands through one of the word ports, consistently G for configuration. Then send point components through their required G/P/Q port. A high-byte write alone does not submit a word. A high-byte read pops one result; read the low latch next.

Limits

There are 256 tagged input words and 256 output words. Nearly-full means input count greater than 240. Point arithmetic is shared: SceneEngine RUN or CPU-stop state owns it. Use the port only after the scenery engine is idle, done or faulted. It has no separate completion IRQ; poll or use markers.

7.2

Point results, flags and numerical limits

Port	Input	Output words
G	X,Z signed words	Camera X,Y,Z,EF (4 words). Ground base and matrix must be initialized.
P	X,Y,Z signed words	Camera X,Y,Z,EF,AZ (5 words). AZ is absolute Z delta.
Q	Camera X,Y,Z words	PX16,PY16,Qdepth (3 words), using the current SceneEngine view parameters.

EF flag, low byte	Meaning
$80	Z negative.
$40 / $20	X < -Z / X > Z.
$10 / $08	Y < -Z / Y > Z.
$02	Transform was halved to fit its fixed-point representation.
$01	Transform overflow. High byte of EF contains the exponent.

Error word bit	Meaning
0	Input overflow.
1	Output read underflow.
2	Protocol error, including invalid command sequence.
3	Point transform overflow.

An empty result read returns $8000 and flags underflow; do not treat it as a valid coordinate. Query gives the full error flags and completion count. Feed only complete commands/points and drain in the same order. Result port addresses all consume the same queue.

Limits

Input coordinates are signed 16-bit. Overflow/scaling flags are part of the result contract, not decoration. Projection can clamp a point to view bounds; this does not clip a line or polygon for you. CPU code using the port must perform geometry clipping and interpret EF before connecting projected points. SceneEngine performs its own multi-point clipping.

Projection setup

The point port uses the view words at $FC:$178B/$178C, described in Section 6.3. Renderer SETPROJ does not configure it. After a point-port reset, reload matrix, eye, base and view as required by your application; do not depend on another application's retained state.

8.1

OS-9 access and layer setup

Use current **grfdrv256 SS.Gpu ($D6)** on a live terminal path. A is the path number; X high byte selects a function and X low byte is its block/flags/command. I$SetStt=$8E, I$GetStt=$8D. Carry set reports error B. Protect the 6809 static U before using U as an argument or return value.

Function	Registers	Operation
0 LPk8, Get/Set	XL=physical block, Y=offset, U=byte	Read/write mapped byte; Get returns U.
1 LPk16, Get/Set	XL=block, Y=offset, U=word	High byte first; Get returns U.
2 LPkN, Set	XL=block, Y=request, U=count	Request: offset word, mask byte, zero byte, data. Write offset+(i & mask), count 1..$1FFC.
3 Scene, Set	Y=caller buffer, U=1..$2000 bytes	Upload at that buffer's logical address. Use explicit SE_ADDR/DATA for a portable target address.
4 Rec, Set	Y=8-byte units, U=1..1024 units	XL bit 0 legacy 480 adaptation; bit 1 wait for SceneEngine idle. Driver checks FIFO room per unit.
5 SceneL, Set	Y=range list	Pairs of start,end-exclusive words; end=0 terminates. Uploads caller memory at those logical addresses.
6 SeOut, Get	Y=buffer, U=1..$1000 word capacity	XL 0 drain, 1 GO, 2 GO-AT, 3 RESUME. Return U=words, X=status. Repeat if still busy.

Show the GPU

```
SS.Layer $8E: SetStt A=terminal, X=layer 0..2, Y=source 3
SS.MCR   $8C: GetStt returns X=low byte, Y=high byte
SS.MCR   $8C: SetStt X=low byte, Y=high byte ($FF leaves unchanged)
```

Set low MCR bits 2 and 3 (graphics + bitmap). Text may overlay the picture; the example turns text off while displaying, then restores the saved MCR. It selects 60 Hz with high byte zero. Layer 0 is nearest. SS.Layer has no matching readback service in this driver, so the examples require a fresh terminal and restore layer 0 to source 0.

The driver maps physical blocks in its own scratch window. Do not write the process MMU directly from a normal OS-9 application. Direct hardware addresses in this manual are for a suitably mapped bare-metal environment or driver, not an invitation to POKE those CPU addresses under OS-9.

8.2

Batch calls, synchronization and IRQ

Function	Registers / item	Behavior
7 FixN, Set	Y=request like LPkN. XL=0 fixed VICKY, XL=1 point port.	VICKY offsets 0..31 map $FFC0-$FFDF; count 1..32. Point offsets 0..7 map $FED8-$FEDF; count 1..8.
8 Pp, Get	Y=items, U=total bytes. Item: cmd,port,wordsIn,wordsOut,data.	Runs point-port transactions; output replaces input storage. Allocate enough room for the larger word count.
9 List, Get/Set	Y=7-byte items, U=count 1..255. Item: kind b,X w,Y w,U w.	Kind 0 Set, 1 Get, 2 Wait, $FF no-op. Wait subfunction 0 raster, 1 FIFO free. No nested List.

LPk8/LPk16 renderer writes reserve queue room; reset waits for idle. Bulk LPkN does not automatically make an arbitrary long upload safe: batch within available FIFO space and poll. SeOut may return with the engine still busy after filling the buffer or its bounded wait; continue draining with command zero.

Work is executed in FIFO order.



SWAP publishes at vertical blank; FENCE reports earlier work complete.

The CPU must keep room for uploads, register writes and complete records.

Drain SceneEngine OUT even when drawing records use SEPORT.

Completion IRQ

GPU completion-counter changes and SceneEngine transitions from RUN to CPU/DONE/FAULT generate an interrupt pulse. Their OR is VICKY interrupt source 2, mapped to **INT_PENDING_1 bit 1** ($FE21). The corresponding control registers are polarity $FE25, edge $FE29, and mask $FE2D. Pending bits acknowledge by writing ones.

The interrupt source is shared. A handler must inspect both engines and distinguish completion from sticky parser errors. Under OS-9, integrate with the kernel/driver interrupt ownership; the examples use polling and do not modify global IRQ mask/control registers.

Limits

FIFO-empty is not a completed frame. FENCE does not display a frame. SWAP waits for vertical blank and publishes the back frame. Multi-unit commands must remain contiguous, and CPU records must not interleave with a live SceneEngine adapter stream. One application must own this global hardware at a time.

8.3

Feature limits: quick reference

Feature	Limit / programmer obligation
Display	640x240x4, 320x240x8, 320x480x8. No 640x480 renderer mode. Select graphics/bitmap gates and a layer source.
Buffer access	Front/back color and depth are internal. No general CPU framebuffer/depth read port; probe is a write-observation aid.
Primitive coordinates	Integer commands use X 0..639; POLY X/Y signed 16-bit sixteenths. Valid viewport still clips coverage.
POLY	3..8 vertices per command, convex fan. Concavity, holes and antialiasing are not implemented.
Mesh	64 KB, 4,096 combined face/vertex entries. Triangular faces only; CPU supplies normals.
Texture	256 KB total; 256 descriptors. Indexed raw bytes, texel 0 transparent, nearest neighbor, no compression/mipmaps.
UV	Unsigned 8.8; full coordinate range addresses 256 texels per axis. Descriptor encoding alone does not widen UV.
Shade	Two 8 KB banks; 256 materials x 32 levels. One directional diffuse light plus ambient. No specular/per-pixel normal shading.
Depth	16-bit packed inverse depth, greater-or-equal test. Saturation near NEAR and finite precision; no stencil.
Blend	No RGB alpha. Index-zero transparency, screen-door discard and ordered haze replacement are distinct.
Scene lists	16 KB scene + 8 KB context. 16 return levels. Unknown opcodes stop for CPU. No instruction watchdog.
Scene polygons	Adapter buffer 32 vertices including clipping output; excess ignored. Direct 4-bit fill, no automatic Gouraud.
Queues	Renderer 512 units; Scene OUT 1,024 words; point input/output 256 words each. Counts may saturate.
Concurrency	Global resources; scene and point share arithmetic. Driver calls are access helpers, not per-process GPU virtualization.
Sprites	480 GPU mode does not borrow sprite RAM. Sprite mirror table is separate at $FC:$1880-$18FF.

8.4

Troubleshooting and source authority

Symptom	Check first
Blank picture	EN; MCR graphics/bitmap bits; layer source 3; nonzero pixels; viewport height/origin; SWAP completed; text overlay.
480 scene fills half height	MODE and MODE480 both set; height high byte; correct Scene view/YMUL; integer records' ninth Y bits.
Wrong colors / lost shading	Graphics CLUT vs GPU shade LUT; DIRECT flag; all 32 material entries; selected mode bank; palette selection/group.
Texture corruption	Descriptor base/dimensions; 8.8 UV order; all UV continuations; upload queue room; asset overlap and address wrap.
No scene completion	Drain CPU OUT; inspect CPU/fault state; valid $007B start; bounded loops; stack depth; do not wait for DONE without draining.
Malformed records	Opcode in byte 7; all payload bytes written; $FE continuations; SETMAT takes five units; no interleaved producer.

Source key used in page footers

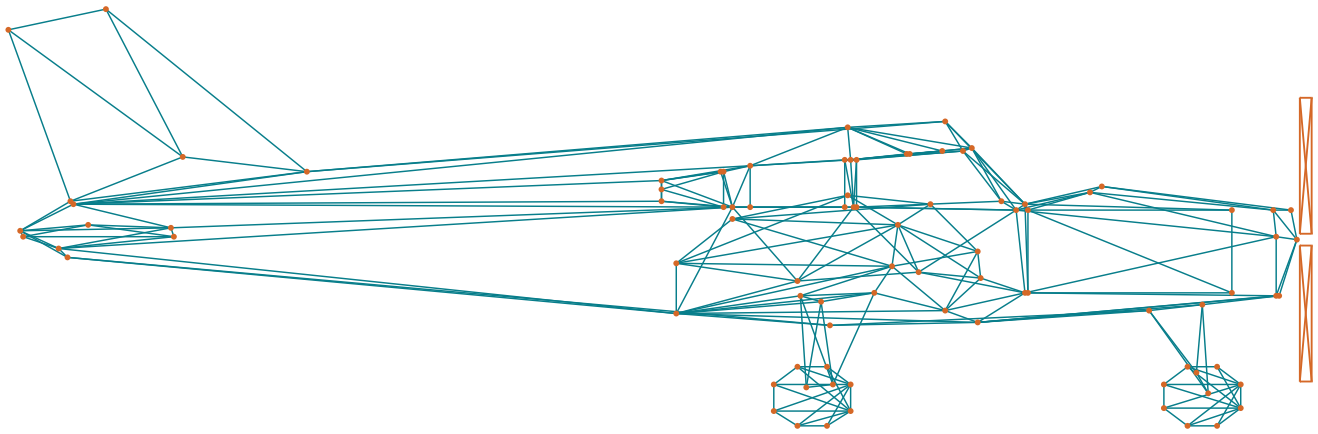
Key	Inspected source under fpga-6809-cores-staging/source, unless stated
[1]	gpu3/gpu3_top.v
[2]	gpu3/gpu3_core.v
[3]	gpu3/gpu3_fb.v
[4]	gpu3/gpu3_tri.v
[5]	gpu3/gpu3_mesh.v
[6]	SceneEngine.v, SceneEngine_units.v, SceneEngine_pt.v, gpu3/gpu3_seport.v
[7]	TinyVKY2K2_IO_Page0_Devices.v, TinyVickyCoreModule.v
[8]	IRQ_Controller_Jr.v
[9]	TyVKYVGE_Pixel_Encoder.v
[10]	nitros9/defs/wildbits.d
[11]	nitros9/level2/wildbits/cmds/grfdrv256.asm

The companion source-manifest.json records the inspected file hashes. Facts in this manual come from these local primary sources. Host models are used only for the limited example checks stated in Section 10.1; they are not authority for unsupported opcodes or old record layouts.

8.5

Flight Simulator II: capability showcase

Flight Simulator II is a useful case study because one frame combines large scenery coordinates, a small moving aircraft, shaded clouds, image textures and a cockpit. These pages trace the inspected FSII implementation and propose repeatable demonstrations. They are not a new hardware benchmark or a claim that every pictured test has been run.



PIPER AIRCRAFT / ORTHOGRAPHIC SIDE VIEW

Cover and side view: vertices and triangle edges decoded from FSII ac.gpu. Orthographic wireframe illustration; no GPU-rendered pixels implied.

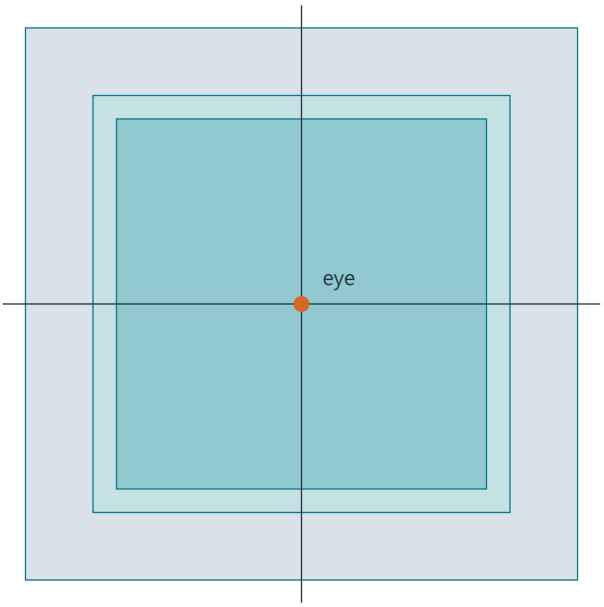
What FSII does	GPU capability it demonstrates
Piper body: 382 triangles; propeller: 24.	Resident mesh rendering and separate transforms, sharing a base of 659 vertices.
Clouds: eight procedural mesh shapes.	Geometry reuse, per-instance matrices, depth tests and interpolated vertex shading.
Buildings and painted surfaces.	Material ramps, indexed image sampling and UV interpolation.
Cockpit around the moving world.	GPU viewport positioning and composition with bitmap artwork; index-zero transparency.
Lights, cloud shadows and propeller.	Application animation and deliberate half-coverage effects using ordinary draw records.

Read the evidence labels

SOURCE means behavior present in the inspected FSII code/data. ILLUSTRATION means an explanatory drawing. TRY means a proposed observation to make on a K2, with an expected result and a limit to watch.

8.6

Distance lab: kilometers, miles and map units



16.128 km: draw bound

18.176 km: population bound

24.064 km: retirement bound

Cloud distance = max(|dx|, |dz|)

These are square bounds in map space, not circles of equal radial distance.

One map unit = 256 meters.

Axes: horizontal X; vertical Z.

Source setting	km	statute mi	nautical mi
Nominal scenery setting	20.000	12.427	10.799
Scenery map-unit threshold	19.968	12.408	10.782
Cloud draw threshold	16.128	10.021	8.708
Cloud population threshold	18.176	11.294	9.814
Cloud retirement threshold	24.064	14.953	12.994

SOURCE: VISIBILITY_KM=20 becomes VisUnits=78 by integer division; 78 map units is 19.968 km. It controls the cited object/scenery tests, not every ground shape or radio station. Cloud distance uses max(|dx|,|dz|); a corner of its drawn square lies farther away in Euclidean distance than an axis point.

One statute mile is 1.609344 km; one nautical mile is 1.852 km. DME uses nautical miles. Do not compare a DME number directly with a map-axis threshold in kilometers and conclude that GPU clipping is wrong.

These are FSII policy thresholds, quantized to map units, not the GPU's universal maximum drawing range. Translation guard capacity is about +/-65.536 km per axis in this path; it is a numerical guard, not guaranteed visibility.

8.7

Cloud laboratory: shape, shade and reuse



8 resident shapes feed a moving population of cloud instances.

ILLUSTRATION: schematic faceted clouds show the reuse idea, not the exact generated cloud topology.

SOURCE	Implementation and limit
Shape budget	8 shapes x (56 triangular faces + 30 vertices) = 688 mesh entries, or 11,008 bytes. Instances reuse these shapes.
Population	Target 24 clouds inside the population bound; 48 slots include clouds retained in the outer band. This does not promise 24 simultaneously visible clouds.
Lighting	Smoothed vertex normals and GOURAUD interpolate a shade scalar; white/light-gray material ramps supply the final indexed colors.
Coverage	Flags $17 = ZTEST | ZWRITE | GOURAUD | SCREENDOOR. The regular half-coverage pattern is intentional; colored specks from wrong palette entries are not.
Weather and shadows	Two configured cloud layers and altitude-dependent wind drive placement/drift. Current ground-texture path draws cloud shadows; farmland/meadow tile draws are commented out.

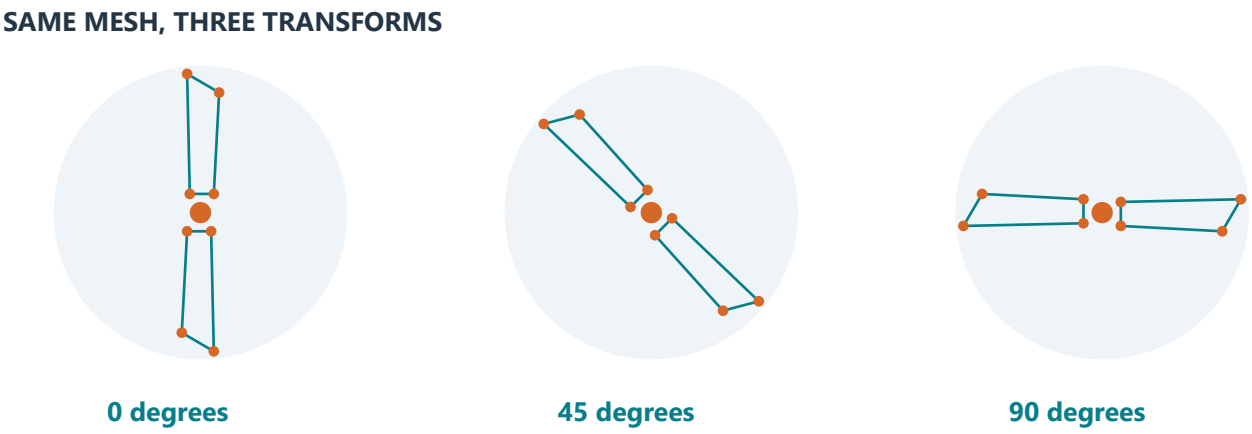
TRY: pass beneath one cloud

Observe its silhouette, shade and ground shadow as the eye moves. Expected: a faceted volume with a smoothly changing shade ramp, clipped by nearer geometry. Limit: screen-door coverage is not RGB alpha, soft volumetric light scattering or a filtered shadow map.

Authoring lesson: spend mesh memory on reusable shapes, then vary the matrix and position. FSII keeps population and wind policy in software; the GPU receives transforms and draws the submitted geometry.

8.8

The propeller: a tiny scene hierarchy



Illustrative front view: orientation samples, not captured frames or RPM timing.

SOURCE: the airframe and propeller are separate mesh ranges. The propeller vertices are centered on their hub. FSII draws the body with M,T, then changes the transform for the propeller. Neither the airframe nor the propeller vertex array needs rewriting for each angle.

```
Body:  SETMAT(M, T);              MESH(body)
Prop:  SETMAT(M * Rz(angle),
              T + transformedHub); MESH(prop)
```

Quantity	Meaning
Hub in asset units	(0,-12,215) in 1/64-meter units: (0,-0.1875,3.359375) meters relative to the body.
Geometry	382 body triangles + 24 propeller triangles. Shared vertex base; independent transforms.
Animation phase	AcRnd advances by min(vRpm,63) x 256 modulo 65,536 per rendered update. Zero RPM leaves the phase still.
Example increments	Gauge value 13: 18.28125 degrees/update. Value 49: 68.90625. Cap 63: 88.59375.
Important limit	This is a display animation step, not time-integrated physical RPM. Capture cadence can still make a rotating prop appear slow or stationary.

TRY: stopped, idle, high throttle

Keep a fixed spot view. A stopped prop should hold its angle; increasing RPM should advance its phase while the hub stays attached to the nose. A wandering hub points to transform composition, not a reason to move the body vertices.

8.9

Airport flythrough: texture, light and layers

TRY / observation	What it demonstrates / limit
Orbit a hangar at a fixed distance.	UVs stay attached to the walls and roof as perspective changes. Nearest-neighbor indexed textures can shimmer or alias at distance; there are no mipmaps.
Bank the Piper in spot view.	The 16-color material ramps represent lighting; GPU shade rows map 32 levels onto these colors. Smooth shading does not add geometric detail or round a coarse silhouette.
Inspect lights at dusk.	Left red, right green and tail white use constant shade rows for emissive-looking lamps. This is an application material convention, not a new light source that illuminates neighboring geometry.
Watch the beacon and landing beam.	Timed visibility/material choices animate the beacon; a screen-door polygon represents the landing beam. Neither implies volumetric light or RGB blending.
Move scenery behind the panel.	The configured GPU viewport fits the cockpit opening, while bitmap artwork remains stable. A correct 480-row setup must align origin, height and projection scaling.
Approach from about 20 km out.	Log eye position, object position and the actual range predicate. Compare with the 19.968 km map-unit threshold; do not treat every visible object as governed by that one test.

A useful proof-of-capabilities capture

Make a short sequence: airport approach, cloud crossing, banked spot view, stopped-to-spinning propeller, then dusk lights. Caption each shot with core filename, FSII asset revision, 240/480 mode, palette bank, dither state and units. Include a still frame beside motion so geometry, palette and cadence can be judged separately.

Keep the claim precise

The showcase is grounded in inspected source and assets. It supplies reproducible observation ideas, not measured frame-rate or timing guarantees. A source constant does not establish the result on a particular flashed core.

Source key extension: [12] nitros9/level2/wildbits/flightsim2/port/fs_wb.d, fsx.asm, fsx_gpu.d, fst.asm, fscl.asm, fswc.asm. [13] cloud/range routines in that same tree. [14] fs2 ac.gpu and wildbits-buildkit/tools/flightsim2/aircraft/ac2gpu.py. Hashes are in the companion manifest.

8.10

The real aircraft asset, in three views

These images use the aircraft mesh and palette bytes from the inspected FSII ac.gpu, drawn by the existing host GPU reference model. They show the same body and separate propeller under three illustrative camera/attitude combinations. No replacement aircraft model or AI-generated rendering is used.



HOST-MODEL IMAGES. Actual FSII geometry and 240 uploaded palette entries; 320-pixel rendering enlarged without filtering. These are not captures from the K2 or measurements of frame rate.

What to inspect

Follow the wing taper and fin outline as the attitude changes. Compare the white fuselage, blue wing tops, dark stripe and propeller. Geometry determines the silhouette; interpolated shade and the material ramps determine the surface colors. If the outline is coarse, extra shade levels cannot add vertices.

A useful next experiment

Hold the mesh fixed and vary only the light, then only the matrix, then only the palette. Keeping two stages fixed while changing one makes it easier to distinguish geometry, shading and palette problems. The companion manifest records the exact asset hash used here.

9.1

Glossary: A-M

Term	Meaning / section
Adapter	Converts SceneEngine drawing words into renderer primitives. 1.1, 6.10.
Back buffer	Color buffer being rendered, published by SWAP. 3.3.
Bayer matrix	Ordered 4x4 threshold pattern used for dithering/haze. 5.1-5.4.
CLUT	Color look-up table: index-to-RGB graphics palette. Different from shade LUT. 5.1.
Corner 0 / fan	First vertex anchors triangles (0,i,i+1); choose a valid fan or split. 3.5-3.7.
Context	Private variables/camera/workspace used by SceneEngine. 6.2.
Continuation	Renderer unit tagged $FE, carrying seven more payload bytes. 3.1.
CPUSTOP	Scenery state requiring host instruction handling and RESUME. 6.4, 6.10.
Decal	Drawn in stream order without depth operations in the ground polygon path. 6.5.
Depth key	Packed inverse depth: larger is nearer. 4.3.
DIRECT	Flag that bypasses shade lookup and uses material/texel as color index. 3.4, 5.2.
Dither	Spatial selection among quantized shade levels. Not edge antialiasing. 5.1.
Face	Triangular mesh record with indices, material, texture, flags and normal. 4.2.
FENCE	Ordered completion marker; does not swap buffers. 3.3.
FIFO	First-in, first-out queue; software must obey capacity. 2.3.
Fixed point	Integer whose implied binary scale represents a fraction. 4.1, 5.4.
Gouraud	Interpolation of a vertex shade scalar across a triangle. 5.2.
Guard band	Larger clip rectangle used before final viewport raster clipping. 4.2.
Haze	Distance-key-based ordered replacement by one palette color. 5.4.
Kernel	Points from which the entire polygon interior is visible. 3.5.
Material	Index selecting one row of the GPU shade table. 5.1.
Mesh	Uploaded face/vertex data transformed and drawn by MESH. 4.2.

9.2

Glossary: N-Z

Term	Meaning / section
Nautical mile	1.852 km; distinct from a 1.609344 km statute mile. 8.6.
Normal	Q1.15 direction used for directional diffuse lighting. 4.2, 5.2.
Opcode	Numeric instruction tag: byte 7 in renderer units, byte 0 in scenery. 3.1, 6.4.
Origin	World-to-local reference used by scenery coordinate conversion. 6.8.
Outcode	Flags describing a transformed point relative to clip planes. 7.2.
Palette bank	One 256-entry RGB graphics CLUT; four physical banks. 5.1.
Perspective correction	Adjusting UV interpolation by inverse depth; segmented here. 5.4.
Point port	Shared G/P/Q transform and projection interface at $FED8-$FEDF. 7.1.
Q	Packed inverse-depth key in renderer vertices; also the name of the projection port. 4.3, 7.2.
Rasterizer	Hardware that converts triangle coverage and attributes into fragments. 3.4.
SCREENDOOR	Discard on alternating X/Y parity for half coverage. 3.4.
SEPORT	GPU_CTRL bit 7, routes scenery drawing directly to GPU. 2.2, 6.10.
Shade LUT	Material and shade-level to palette-index table. 5.1.
Sprite	Independent display object system; not GPU mesh or scenery geometry. 8.3.
SWAP	Publish the rendered back frame at vertical blank. 3.3.
Texel	One indexed image sample, one byte in texture memory. 5.3.
Texture descriptor	Four-byte base/dimension/wrap-clamp record. 5.3.
UV	Unsigned 8.8 texture coordinates in texels. 5.4.
Vertex	Position plus depth/shade/UV or mesh normal information. 3.4, 4.2.
Viewport	Local rendering area with configured height and display row origin. 1.2.
Winding	Order around the boundary; clockwise screen area is positive with Y downward. 3.7.
Writeback	SceneEngine output of modified context bytes before a stop. 6.10.

10.1

Small demos: requirements and support

This edition reduces the drawing to **one rectangle, then one scenery triangle**. The assembly application is 54 lines and BASIC09 is 23 lines. Repeated OS-9 setup, queue handling and cleanup live in supplied support source, so the printed examples emphasize the data and draw sequence.

Before running	Reason
Fresh live K2 terminal; no GPU owner.	Support checks GPU_ID=$34, enable/busy state and SceneEngine status before taking ownership.
Current grfdrv256; BASIC09 needs SYSCALL.	All physical block access goes through the driver. No direct process-MMU POKEs.
Existing CLUT 0 indices 1,9,14 visible.	The examples leave palettes unchanged; preview RGB colors are illustrative.
ENTER after rectangle; ENTER after triangle.	First ENTER advances; second restores MCR, returns layer 0 to bitmap 0 and exits.

Support is source, not a hardware feature

gpuvars.inc and **gpuos9.inc** are ordinary LWASM source includes. **gpulib.b09** is a BASIC09 procedure loaded alongside gpudemo. They are supplied in full and have no firmware installation step. The small public program uses these files; it is not standalone without them.

Support routine	Contract
gpu_open	Own idle GPU; save MCR; select 320x240, CLUT 0, no haze, row origin 0, height 240. Preserve static U.
records / idle	Send complete eight-byte records through SS.Gpu with FIFO checks; bounded completion polling.
gpu_scene	Clear back frame; upload identity context and the one-triangle list; set view and YMUL; GO-AT through SEPORT; drain OUT until DONE.
cleanup / gpulib	Abort/flush owned scene work, reset/disable GPU, restore MCR, assign layer 0 to bitmap 0. Do not overwrite a pre-existing GPU owner.

Validation boundary

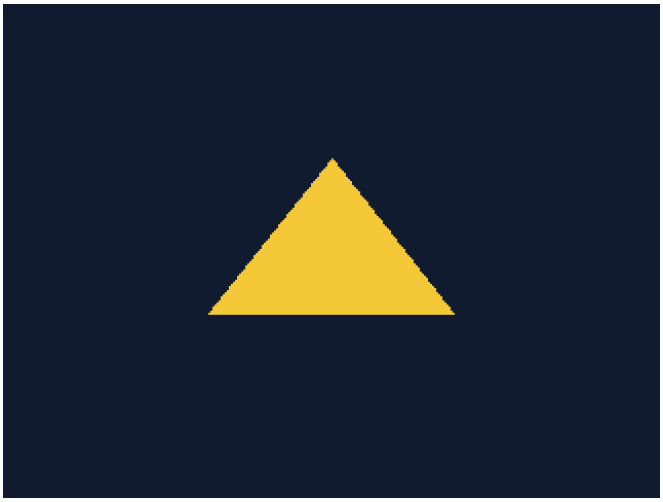
The assembly application and its includes assemble with LWASM. BASIC09 has static structural checks only. The one-triangle scenery list and the rectangle records pass host-model checks. No physical K2, OS-9 runtime, core build or disk change was performed for this manual.

10.2

Two small pictures and how to run them

Rectangle: three units	Scene triangle: 29 bytes
CLEAR index 1, RECT index 9, SWAP.	COLOR 14, PBEGIN, MOVE3, DRAW3, DRAW3, PEND, END.
X=160..480, Y=70..170.	Three world points at Z=256; identity camera.





Host-model illustrations, not K2 captures. Triangle shade is constant; optional Gouraud and texture data follow in 10.5.

```
Assembly:  load gpudemo
gpudemo

BASIC09:
B: load gpulib.b09
B: load gpudemo.b09
B: run gpudemo
```

Use path 0 for ENTER and live terminal path 1 for driver calls. Stay on that terminal until exit. Assemble with the companion build_examples.ps1: LWASM, --6809, --format=os9, and the current WildBits definition paths. Keep both .inc files beside gpudemo.asm.

Visibility in the printed assembly

SS.Layer ($8E) with X=0,Y=3 selects GPU on layer 0. SS.MCR ($8C) with X=$0C,Y=0 enables graphics and bitmap output while hiding text. gpu_open sets viewport height 240 and origin 0; publish submits SWAP and waits. BASIC09 gpulib performs the same sequence.

10.3

6809: rectangle, then a scene triangle (1/1)

gpudemo.asm - source lines 1-54; omit the left line numbers when copying.

```
001   nam gpudemo
002   use defsfile
003   mod eom,name,Prgrm+Objct,ReEnt,start,size
004  name fcs /gpudemo/
005   fcb 2
006   org 0
007   use gpuvars.inc
008  size equ .
009  start lbsr gpu_open
010   lbcs fail
011   leay shapes,pcr
012   ldd #(shapes_end-shapes)/8
013   lbsr records
014   lbcs fail
015   lbsr publish
016   lbcs fail
017  * Layer 0 selects source 3; enable graphics + bitmap.
018   ldx #0
019   ldy #3
020   ldb #$8E
021   lbsr setstat
022   lbcs fail
023   ldx #$0C
024   ldy #0
025   ldb #$8C
026   lbsr setstat
027   lbcs fail
028   lbsr enter
029   lbcs fail
030   lbsr gpu_scene
031   lbcs fail
032   lbsr publish
033   lbcs fail
034   lbsr enter
035   lbcs fail
036   lbsr cleanup
037   clrb
038   os9 F$Exit
039  publish leay swaprec,pcr
040   ldd #1
041   lbsr records
042   bcs pubdone
043   lbsr idle
044  pubdone rts
045  shapes
046   fcb $00,$00,$00,$00,$01,$00,$01,$03,$00,$A0,$01,$E0
047   fcb $46,$AA,$09,$02
048  shapes_end equ *
049  swaprec
050   fcb $00,$00,$00,$00,$00,$00,$00,$08
051   use gpuos9.inc
052   emod
053  eom equ *
054   end
```

10.4

BASIC09: the same two small pictures (1/1)

gpudemo.b09 - source lines 1-23; omit the left line numbers when copying.

```
001  PROCEDURE gpudemo
002  (* Live K2 terminal; SYSCALL and gpulib required. *)
003  (* ENTER shows the triangle; ENTER again restores display. *)
004  DIM pixels(24),scenepts(29):BYTE
005  DIM i,n:INTEGER
006  FOR i=1 TO 24
007    READ n
008    pixels(i)=n
009  NEXT i
010  FOR i=1 TO 29
011    READ n
012    scenepts(i)=n
013  NEXT i
014  RUN gpulib(pixels,scenepts)
015  END
016  (* CLEAR to index 1; RECT index 9; SWAP. *)
017  DATA 0,0,0,0,1,0,1,3
018  DATA 0,160,1,224,70,170,9,2
019  DATA 0,0,0,0,0,0,0,8
020  (* COLOR 14; PBEGIN; MOVE3; DRAW3; DRAW3; PEND; END. *)
021  DATA 37,0,14,1,120,47,1,255,160,255
022  DATA 192,1,0,2,0,96,255,192,1,0
023  DATA 2,0,0,0,96,1,0,41,121
```

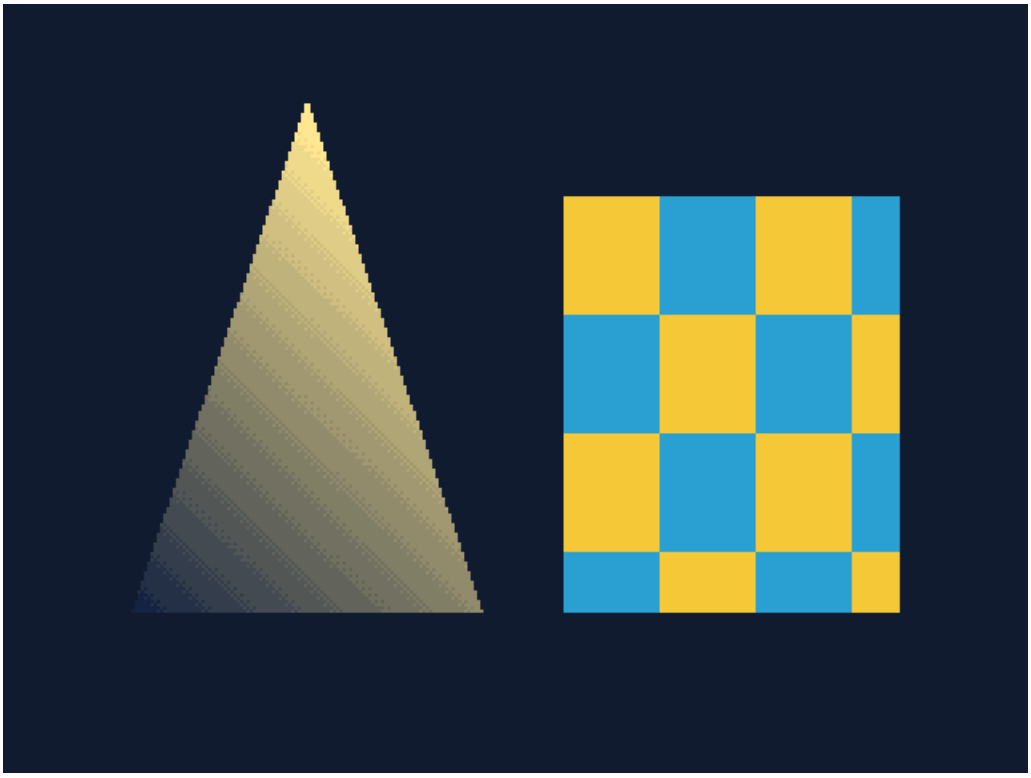
What the short caller provides

pixels is three eight-byte renderer units. scenepts is the complete 29-byte scenery program. gpulib performs setup, submission, visibility, ENTER handling and restoration. Change the data, not the driver calls, for your next small object. The complete support procedure is in the companion archive.

```
Scene points: (-96,-64,256), (96,-64,256), (0,96,256)
Context: identity rotation; eye at zero; IP=$4000
View: CX=5120 CY=1920 SX=1280 SY=480
      RIGHT=10239 BOTTOM=3839; YMUL=$0400
```

10.5

Add Gouraud shading and an image texture



Renderer-model preview: a dithered 16-color Gouraud ramp and an indexed 8x8 checker texture. The next pages give the upload/data recipe; they are extensions to the complete demo, not separate programs.

Step	Action
1. Reserve palette entries	Save CLUT 0 entries 16..31 before changing them. Upload 16 B,G,R,A entries, using the formula below. Restore these bytes on every exit.
2. Load shade row	At upload address $802020, write 32 bytes: 16,16,17,17,...,31,31. This is 320-mode material 1.
3. Load texture	At $000000 write the 64 checker texels. At $C00000 write descriptor 00,00,00,C0 (8x8, clamp U/V).
4. Configure and draw	After normal 320x240 setup, set GPU_CTRL=$4B (EN|MODE|DITHER|TEXEN), CLEAR, submit the 13 units on Section 10.5.3, SWAP and wait.

```
For k=0..15, palette index = 16+k:
  B = floor(65 + 80*k/15)
  G = floor(35 + 195*k/15)
  R = floor(20 + 235*k/15)
  A = 0
CLUT 0 byte address: block $FD, offset $1040 + 4*k
```

10.5.2

Upload helpers: assembly and BASIC09

Use the gpudemo **put8** helper for GPU writes. It preserves X/Y/U and uses a driver call that waits for FIFO room. Here X points to bytes, D is count, and the three UP_ADDR bytes were written first. Add the generated shade-texture.inc to your data area.

```
assetbytes
 std left,u
 ldy #$180D
assetloop
 lda ,x+
 lbsr put8
 bcs assetdone
 ldd left,u
 subd #1
 std left,u
 bne assetloop
assetdone rts

* Example: material 1, 320-mode shade row at $802020
 lda #$80
 ldy #$180A
 lbsr put8
 bcs assetdone
 lda #$20
 ldy #$180B
 lbsr put8
 bcs assetdone
 ldy #$180C
 lbsr put8
 bcs assetdone
 leax shade_row,pcr
 ldd #32
 lbsr assetbytes
```

BASIC09, using helpers 700 and 730 from gpulib

```
off=$180A
val=$80
GOSUB 700
off=$180B
val=$20
GOSUB 700
off=$180C
GOSUB 700
off=$180D
FOR i=0 TO 31
  val=16+INT(i/2)
  GOSUB 700
NEXT i
```

For texels repeat the same address/data pattern with $000000 and 64 bytes; for the descriptor use $C00000 and four bytes. A zero-count call to assetbytes is not allowed. Check carry after each assembly helper call and route failures through your cleanup. The snippets share gpudemo's left variable, so call them between its other upload operations.

10.5.3

Exact shade and texture drawing records

These units are also supplied as shade-texture-records.bin and shade-texture.inc. Each row below is exactly eight bytes in FIFO order. Upload assets first; CLEAR and SWAP are supplied by the surrounding demo.

```
04 01 00 03 00 00 00 10
05 00 0B E0 80 00 00 FE
12 C0 0B E0 80 00 80 FE
0B E0 01 E0 80 00 FF FE
28 00 00 04 00 00 00 10
15 E0 03 C0 80 00 FF FE
23 00 03 C0 80 00 FF FE
23 00 0B E0 80 00 FF FE
15 E0 0B E0 80 00 FF FE
00 00 00 00 00 00 00 FE
07 00 00 00 00 00 00 FE
07 00 07 00 00 00 00 FE
00 00 07 00 00 00 00 FE
```

First four units: GOURAUD polygon, material 1, three vertices with shades 0,128,255. Q=$8000 is constant and depth flags are clear. Next nine units: DIRECT|TEXTURED polygon with four position units and four UV units. Texture ID is zero; UV corners span 0..7 texels.

The 8 x 8 checker image, row-major hexadecimal bytes

```
0E 0E 09 09 0E 0E 09 09
0E 0E 09 09 0E 0E 09 09
09 09 0E 0E 09 09 0E 0E
09 09 0E 0E 09 09 0E 0E
0E 0E 09 09 0E 0E 09 09
0E 0E 09 09 0E 0E 09 09
09 09 0E 0E 09 09 0E 0E
09 09 0E 0E 09 09 0E 0E
```

The checker uses existing palette indices 9 and 14. To shade an image instead, clear DIRECT, assign each nonzero texel a material ID, and initialize that material's 32-entry shade row. Supply per-vertex shades and GOURAUD for an interpolated result.

Palette access and restoration

SS.Palet selects a bitmap's CLUT; it is not the RGB upload operation. Use SS.Gpu LPk8/LPkN with block $FD for these bytes, saving old entries first. In BASIC09, set rr.x=$00FD, rr.y=$1040+i, rr.b=$D6 and call GetStt to save each byte, then SetStt to change it. In assembly use X=$00FD, Y=offset, U=byte with static U protected. Restore all 64 saved bytes before returning to the terminal, including error exits.

10.6

Turn the sample into an application

Goal	Change and obligation
Move the 2D shapes	Change integer coordinates; retain 640-unit X convention in 320 mode. Keep a complete POLY header/vertex sequence.
Move the scenery triangle	Change scene point words or eye/matrix context before GO. Do not write its RAM while busy.
Animate	CLEAR back buffer, submit one complete frame, SWAP, wait; repeat. Preserve asset tables between frames.
Use 480 rows	Set MODE480=1; height high=1, low=$E0. Use the full-480 Scene view with YMUL=$0400, or full-240 view with YMUL=$0800.
Draw 480-row integer lines	Put Y bit 8 into X word bit 15 for each endpoint. The low eight Y bits remain bytes 4/5. Do not use Scene YMUL for these records.
Shade a model	Upload mesh vertices with normals and faces; load shade LUT/CLUT, SETMAT, SETLIGHT, SETPROJ, then MESH. Clear DIRECT.
Texture a scene object	Upload image/descriptor, set TEXEN, insert TEX and UV before vertices, then NOTEX. Routed Scene texture colors are DIRECT.
Combine with a cockpit	Place GPU source 3 in the intended layer, use index-zero transparency, and keep viewport origin/height aligned with bitmap artwork.
Share the machine	Add an ownership protocol, saved context/palette/layer bookkeeping, and driver-integrated completion handling. The demo is intentionally single-owner.

A complete frame is more than drawing commands

A robust application owns the hardware, initializes all state it depends on, handles FIFO backpressure, drains scenery output, checks sticky errors, publishes at a frame boundary, and restores the display it borrowed. Keep those duties even when replacing the demo's shapes with your own scenery or meshes.

Revision record

1.1 - October 27, 2026. K2 PC cover with Piper mesh, corner-0 tutorial, Flight Simulator II capability showcase, shorter examples and shared support sources.

1.0 - October 27, 2026. First illustrated reference for the inspected Houston RC28 interfaces. Includes register maps, renderer/scenery/point formats, shading and texture limits, full 6809/BASIC09 listings, glossary, and section/page index.

I1

Alphabetical index

Topic	Section / printed page
480-line mode	1.2 / p. 7; 3.2 / p. 13; 6.3 / p. 28; 10.6 / p. 57
6809 source	10.3 / p. 52
ABORT	6.1 / p. 26; 6.2 / p. 27
Alpha / transparency	4.3 / p. 21; 5.3 / p. 24
Architecture	1.1 / p. 6
Assets / upload	2.1 / p. 9; 5.3 / p. 24; 10.5.2 / p. 55
BASIC09	8.1 / p. 38; 10.4 / p. 53
Bayer dither	5.1 / p. 22; 5.4 / p. 25
Camera matrix	4.1 / p. 19; 6.2 / p. 27
CLEAR	3.3 / p. 14
Clipping	4.2 / p. 20; 4.3 / p. 21; 6.4 / p. 29
Clouds / shadows	8.7 / p. 44
CLUT / palette	5.1 / p. 22; 10.5.3 / p. 56
Command FIFO	2.3 / p. 11; 3.1 / p. 12
Completion / IRQ	2.3 / p. 11; 8.2 / p. 39
Concavity / kernel	3.5 / p. 16; 3.6 / p. 17
Coordinates	1.2 / p. 7; 3.2 / p. 13
Corner 0 / fill leaks	3.5 / p. 16; 3.6 / p. 17; 3.7 / p. 18
CPUSTOP	6.4 / p. 29; 6.10 / p. 35
Depth / Q	4.3 / p. 21; 7.2 / p. 37
DIRECT flag	3.4 / p. 15; 5.2 / p. 23
Dither	5.1 / p. 22; 5.2 / p. 23
Examples	10.1 / p. 50; 10.2 / p. 51
FENCE	3.3 / p. 14

Page numbers are the PDF page numbers. Section references and the PDF outline provide stable navigation within this edition.

I2

Alphabetical index (continued)

Topic	Section / printed page
Flags / POLY	3.4 / p. 15
Flight Simulator II	8.5 / p. 42; 8.9 / p. 46; 8.10 / p. 47
Frame / SWAP	3.3 / p. 14; 8.2 / p. 39
Glossary	9.1 / p. 48; 9.2 / p. 49
GO / GO-AT / RESUME	6.1 / p. 26; 6.2 / p. 27
Gouraud shading	5.2 / p. 23; 10.5 / p. 54
Graphics visibility	1.3 / p. 8; 8.1 / p. 38
Haze	5.4 / p. 25
HSPLIT	3.3 / p. 14
Image texture	5.3 / p. 24; 5.4 / p. 25; 10.5 / p. 54
Index-zero transparency	1.3 / p. 8; 4.3 / p. 21
Kilometers / miles / range	8.6 / p. 43
Layer setup	1.3 / p. 8; 8.1 / p. 38; 10.3 / p. 52
Limits	8.3 / p. 40
LINE / SPAN / RECT	3.2 / p. 13
LW line width	6.3 / p. 28
Material / shade LUT	5.1 / p. 22
MCR display gates	1.3 / p. 8; 8.1 / p. 38
Memory map	2.1 / p. 9; 6.2 / p. 27
MESH	4.2 / p. 20
Normals / lighting	5.2 / p. 23
Opcode layouts	3.1 / p. 12; 6.4 / p. 29
OS-9 / SS.Gpu	8.1 / p. 38; 8.2 / p. 39
Piper aircraft	8.5 / p. 42; 8.8 / p. 45; 8.10 / p. 47

Page numbers are the PDF page numbers. Section references and the PDF outline provide stable navigation within this edition.

I3

Alphabetical index (continued)

Topic	Section / printed page
Point port	7.1 / p. 36; 7.2 / p. 37
POLY / fan	3.4 / p. 15
Preface	P / p. 5
Probe	2.2 / p. 10; 2.3 / p. 11
Propeller / hierarchy	8.8 / p. 45
Registers	2.1 / p. 9; 2.2 / p. 10; 6.1 / p. 26
Return stack	6.6 / p. 31
Scenery opcode list	6.5 / p. 30; 6.6 / p. 31; 6.7 / p. 32; 6.8 / p. 33; 6.9 / p. 34
SE_AREA	6.8 / p. 33
SE_AREA / SE_ORIGIN	6.8 / p. 33
SE_ATC	6.9 / p. 34
SE_ATC / SE_NAV / SE_RUNWAY	6.9 / p. 34
SE_BITS	6.7 / p. 32
SE_BITS / SE_CMP / SE_RANGE	6.7 / p. 32
SE_CALL	6.6 / p. 31
SE_CALL / SE_END / SE_GOTO	6.6 / p. 31
SE_CALLPTR	6.6 / p. 31
SE_CMP	6.7 / p. 32
SE_COLOR	6.5 / p. 30
SE_COLOR / SE_DOT / SE_DRAW / SE_MOVE	6.5 / p. 30
SE_COLORDUSK	6.5 / p. 30
SE_COLORFF	6.5 / p. 30
SE_COPYW	6.7 / p. 32
SE_COPYW / SE_SETW	6.7 / p. 32

Page numbers are the PDF page numbers. Section references and the PDF outline provide stable navigation within this edition.

I4

Alphabetical index (continued)

Topic	Section / printed page
SE_CULLSLOTS	6.7 / p. 32
SE_DOT3	6.5 / p. 30
SE_DOTSLOT	6.5 / p. 30
SE_DRAW3	6.5 / p. 30
SE_DRAWG	6.5 / p. 30
SE_DRAWSLOT	6.5 / p. 30
SE_END	6.6 / p. 31
SE_GOTO	6.6 / p. 31
SE_LONGLINE	6.5 / p. 30
SE_MOVE3	6.5 / p. 30
SE_MOVEG	6.5 / p. 30
SE_MOVESLOT	6.5 / p. 30
SE_NAV05	6.9 / p. 34
SE_NAV1D	6.9 / p. 34
SE_NEAR	6.8 / p. 33
SE_NEARALT	6.8 / p. 33
SE_NOP	6.6 / p. 31
SE_NOTEX	6.6 / p. 31
SE_ORIGIN16	6.8 / p. 33
SE_ORIGIN32	6.8 / p. 33
SE_PBEGIN	6.5 / p. 30
SE_PBEGIN / SE_PEND	6.4 / p. 29; 6.5 / p. 30
SE_PEND	6.5 / p. 30
SE_RANGE1	6.7 / p. 32

Page numbers are the PDF page numbers. Section references and the PDF outline provide stable navigation within this edition.

I5

Alphabetical index (continued)

Topic	Section / printed page
SE_RANGE2	6.7 / p. 32
SE_RANGE3	6.7 / p. 32
SE_RETURN	6.6 / p. 31
SE_RUNWAY	6.9 / p. 34
SE_SETW	6.7 / p. 32
SE_SLINE	6.5 / p. 30
SE_SLOT3	6.5 / p. 30
SE_SLOTG	6.5 / p. 30
SE_TEX	6.6 / p. 31
SE_TEX / SE_NOTEX / SE_UV	6.6 / p. 31
SE_UV	6.6 / p. 31
SEPORT	6.10 / p. 35
SETLIGHT / SETMAT / SETPROJ	4.1 / p. 19
Shared example support	10.1 / p. 50
Source authority	8.4 / p. 41
Sprites and 480 mode	8.3 / p. 40
Texturing limits	5.4 / p. 25
Troubleshooting	8.4 / p. 41
UV coordinates	5.4 / p. 25; 6.6 / p. 31
Viewport	1.2 / p. 7; 6.3 / p. 28
Winding	3.7 / p. 18
Writeback records	6.10 / p. 35

Page numbers are the PDF page numbers. Section references and the PDF outline provide stable navigation within this edition.